

Ejecución Simbólica Dinámica

Valeria Bengolea

Departamento de Computación

Facultad de Ciencias Exactas, Físico-Químicas y Naturales

Universidad Nacional de Río Cuarto

**Generación Automática de Test basada en
Ejecución Simbólica Dinámica**

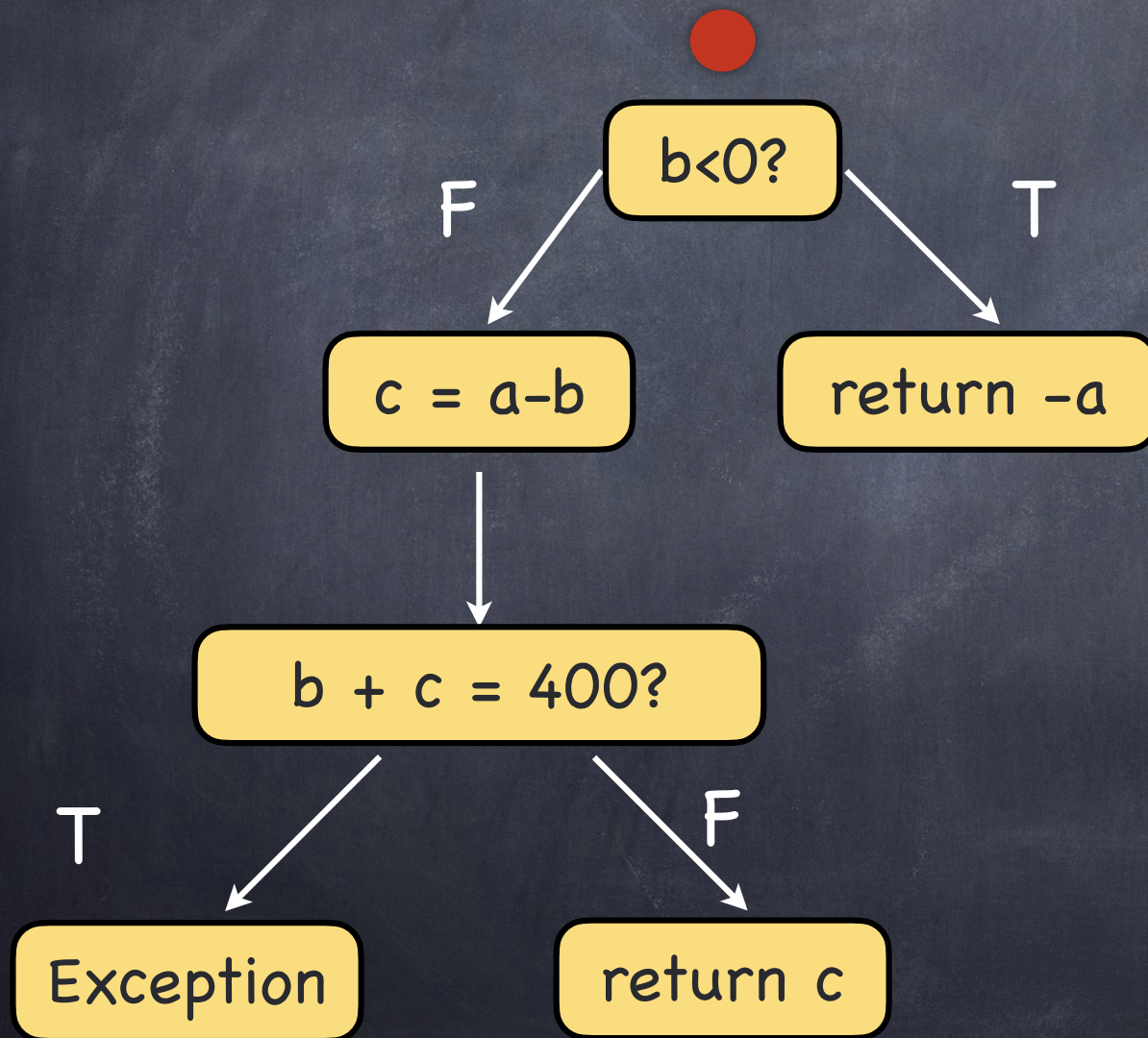
Ejecución Simbólica (SE)

Ejecución Simbólica (SE)

```
public int testMe(int a, int b) {  
    if(b < 0) {  
        return -a;  
    }else {  
        int c = a - b;  
        if (b + c == 400) {  
            throw new IllegalArgumentException();  
        }else  
            return c;  
    }  
}
```

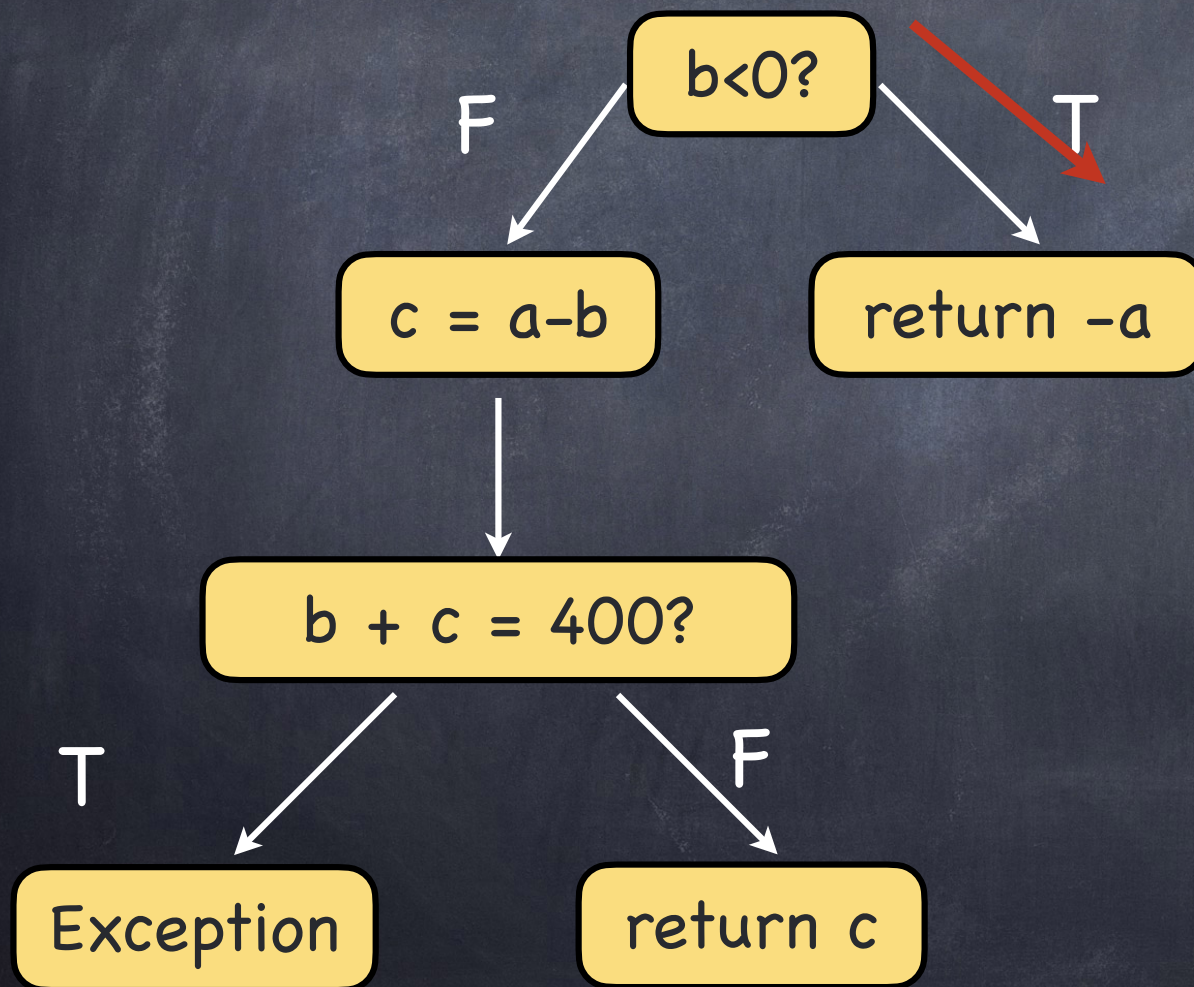

Ejecución Simbólica (SE)

Ejecución Simbólica (SE)



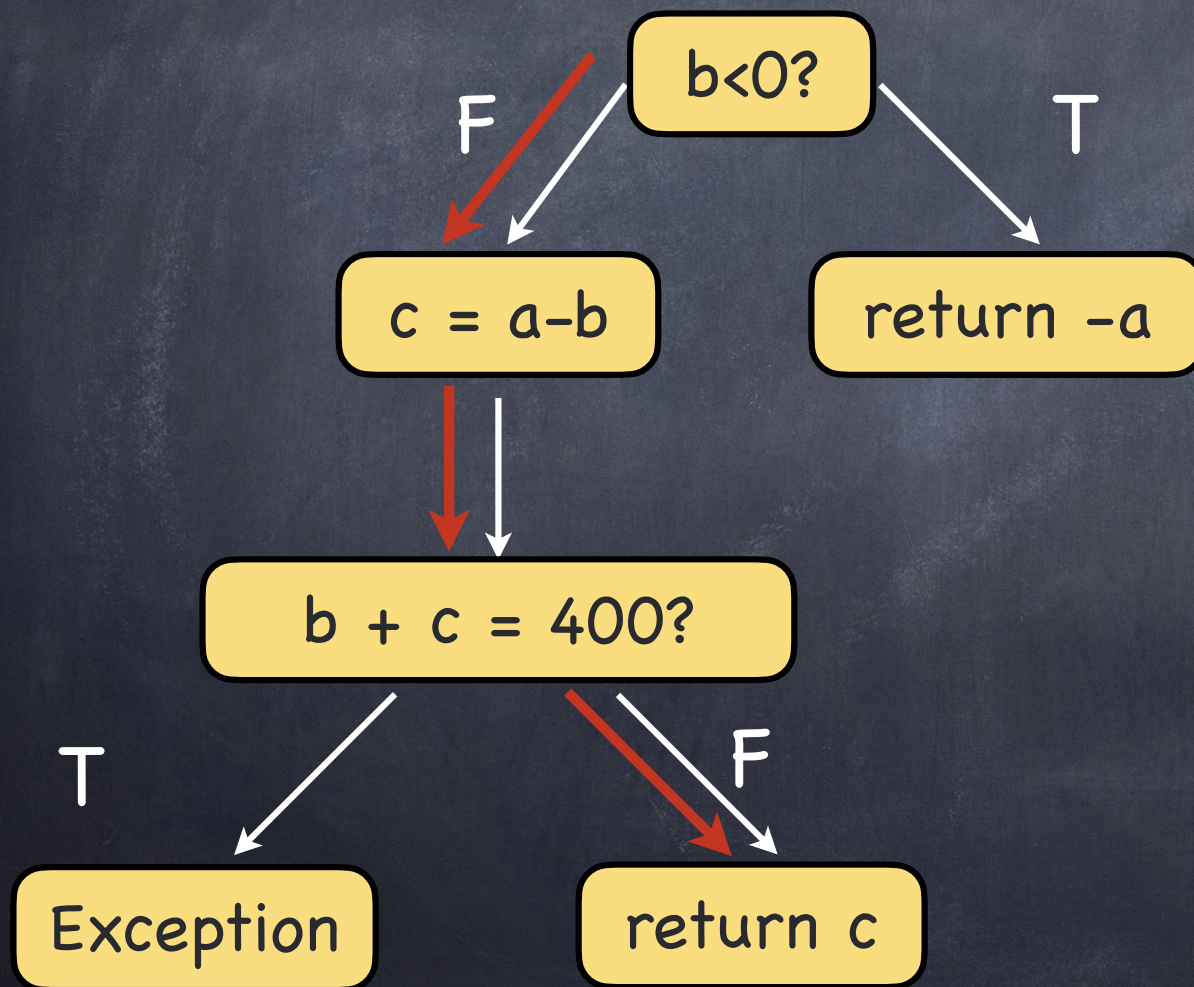
Estado
simbólico:
 $a=a_0$, $b=b_0$
path condition:
true

Ejecución Simbólica (SE)



Estado
simbólico:
 $a = a_0, b = b_0$
path condition:
 $b_0 < 0$

Ejecución Simbólica (SE)

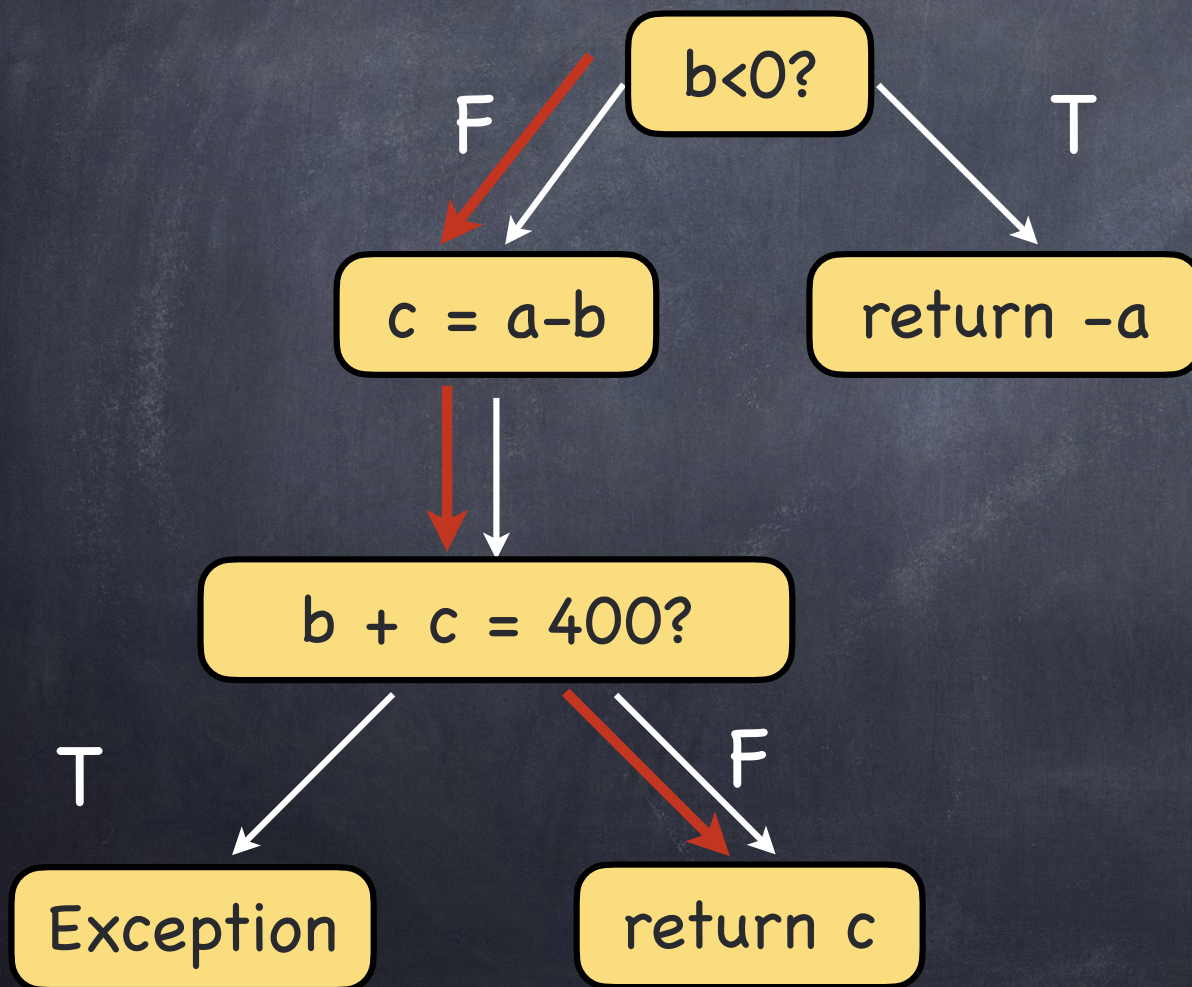


symbolic state:
 $a = a_0, b = b_0, c = a_0 - b_0$

path condition:
 $b_0 \geq 0$

$b_0 + a_0 - b_0 \neq 400$

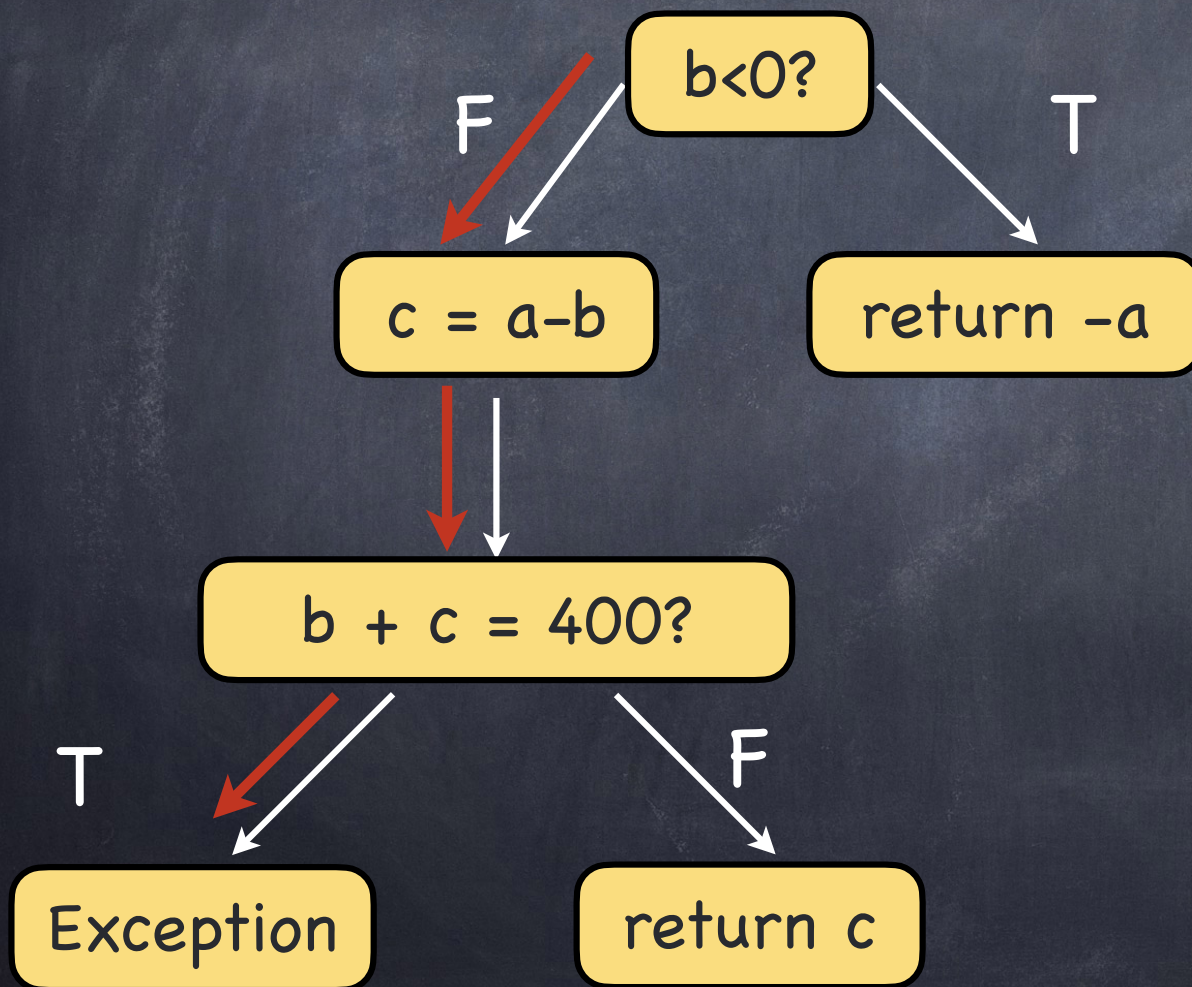
Ejecución Simbólica (SE)



symbolic state:
 $a = a_0, b = b_0, c = a_0 - b_0$

path condition:
 $b_0 \geq 0 \wedge a_0 \neq 400$

Ejecución Simbólica (SE)

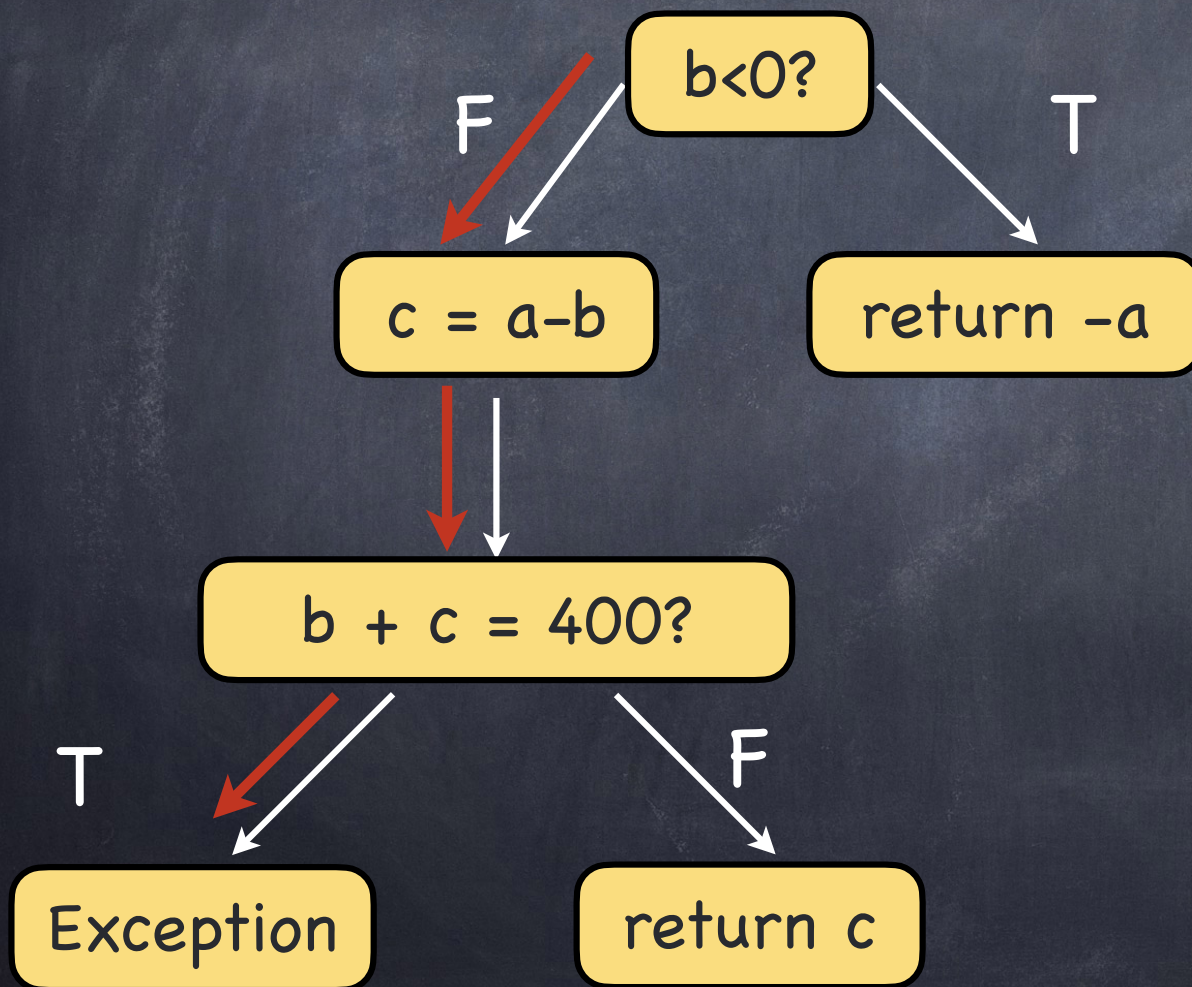


symbolic state:
 $a = a_0, b = b_0, c = a_0 - b_0$

path condition:
 $b_0 \geq 0$

$b_0 + a_0 - b_0 = 400$

Ejecución Simbólica (SE)



symbolic state:
 $a = a_0, b = b_0, c = a_0 - b_0$

path condition:
 $b_0 \geq 0 \wedge a_0 = 400$

Constraint Solver

$$b0 < 0$$

$$b0 \geq 0 \wedge a0 \neq 400$$

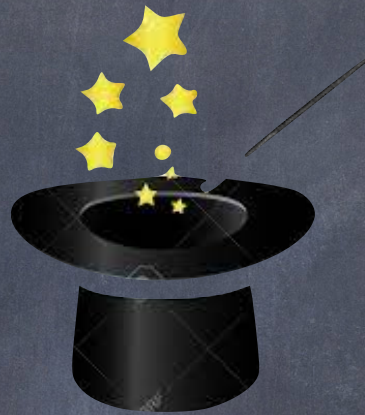
$$b0 \geq 0 \wedge a0 = 400$$

Constraint Solver

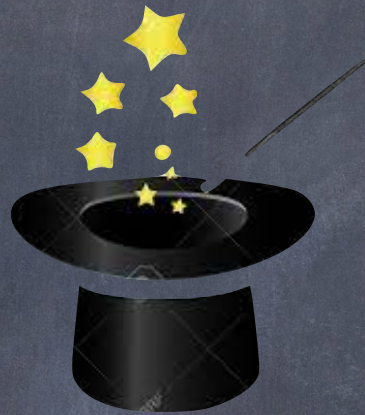
$b0 < 0$

$b0 \geq 0 \wedge a0 \neq 400$

$b0 \geq 0 \wedge a0 = 400$



Constraint Solver



$b0 < 0$

$\rightarrow b0 = -1$

$b0 \geq 0 \wedge a0 \neq 400$

$\rightarrow b0 = 0 \quad a0 = 0$

$b0 \geq 0 \wedge a0 = 400$

$\rightarrow b0 = 0 \quad a0 = 400$

Constraint Solver

Constraint Solver



Constraint Solver



Un programa que resuelve
fórmulas lógicas

SAT-Solver

Input: Una fórmula proposicional (en formato CNF)

Output: SAT/UNSAT

Decidir si una fórmula proposicional es sat/unsat es decidable

Tarda peor caso $O(2^n)$

SAT-Solvers usualmente usan DPLL (Davis- Putnam-Logemann-Loveland)

SMT-Solvers

El SAT-Solver únicamente resuelve fórmulas proposicionales.

Un SMT-Solver permite resolver fórmulas

Con cuantificadores (para todo, existe)

Con enteros, arreglos, números reales, strings, etc

SMT-Solvers

Pero decidir si una fórmula FOL es sat/unsat **NO** es decidable

Input: Una fórmula FOL con teorías (arrays, enteros, números reales, funciones no interpretadas, etc.)

Output: SAT/UNSAT/UNKNOWN

Usa DPLL + solventes específicos para otros problemas

Ejecución Simbólica

```
execution_paths = compute_paths(cfg, limit)
For each cfg_path in execution_paths

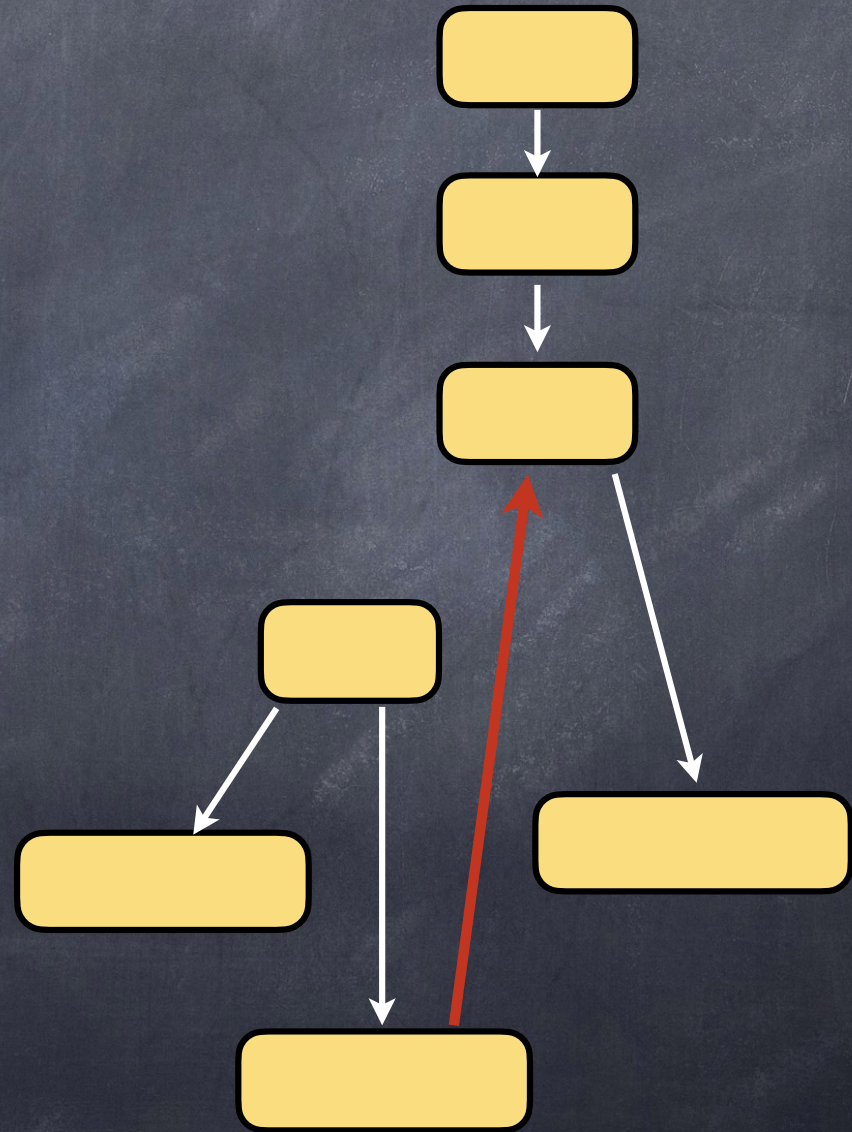
    If budget is not empty
        path_condition = exec_symbolic(cfg_path)
        solution = solve(path_condition)

        if solution is SAT:
            new_test = create_test(entry_f, solution)
            add new_test to testSuite

return testSuite
```


Ejecución Simbólica

Limitaciones



Ejecución Simbólica

Limitaciones

Explosión combinatoria (o incluso infinita) de caminos en el CFG para evaluar

Limitaciones del Constraint Solving

String handling, aritmética no-lineal, aritmética de punto flotante, etc.

Información en Runtime (files, sockets, native code, etc.)

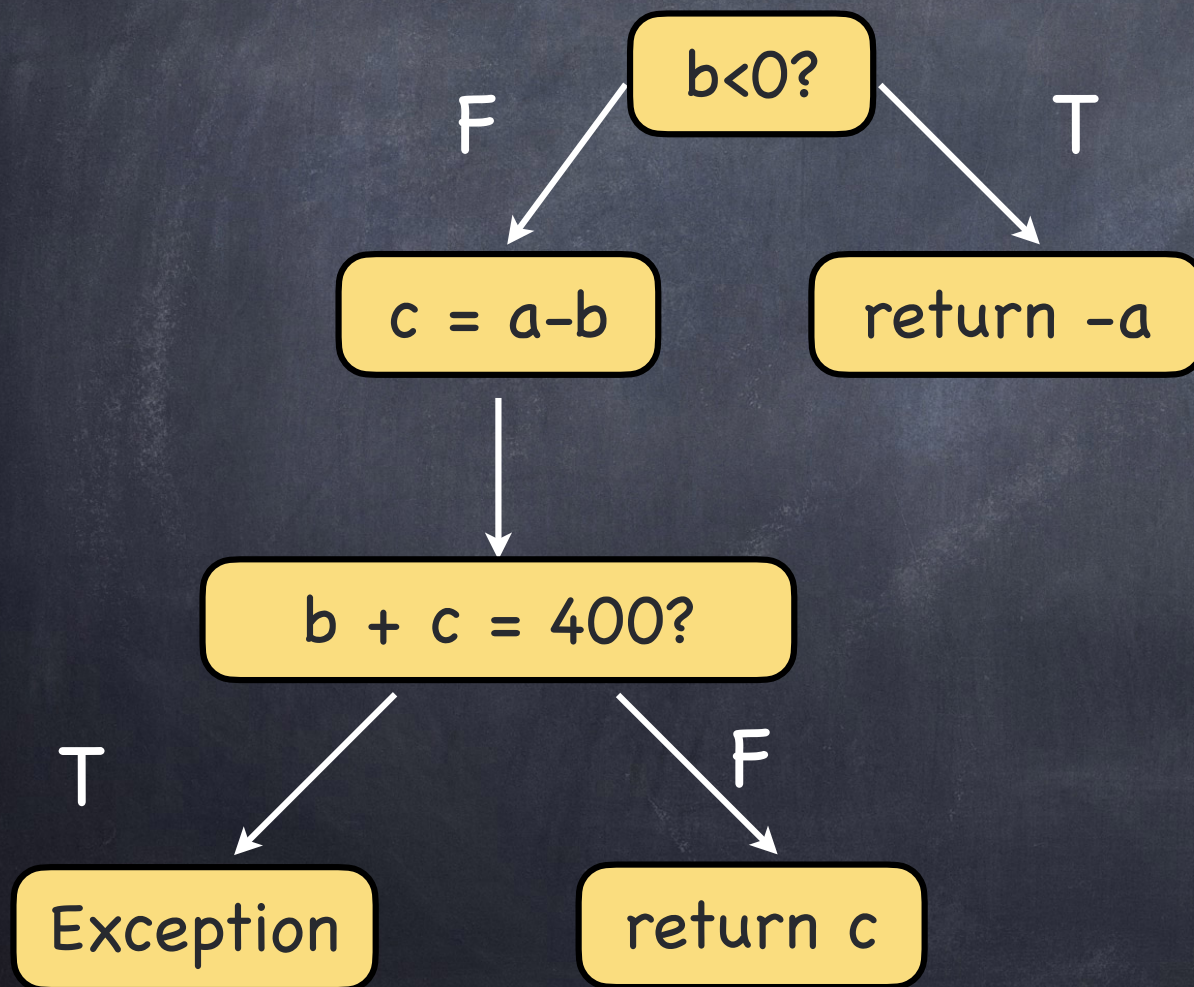
Ejecución Simbólica Dinámica (DSE)

Ejecución Simbólica Dinámica (DSE)

```
public int testMe(int a, int b) {  
    if(b < 0) {  
        return -a;  
    }else {  
        int c = a - b;  
        if (b + c == 400) {  
            throw new IllegalArgumentException();  
        }else  
            return c;  
    }  
}
```

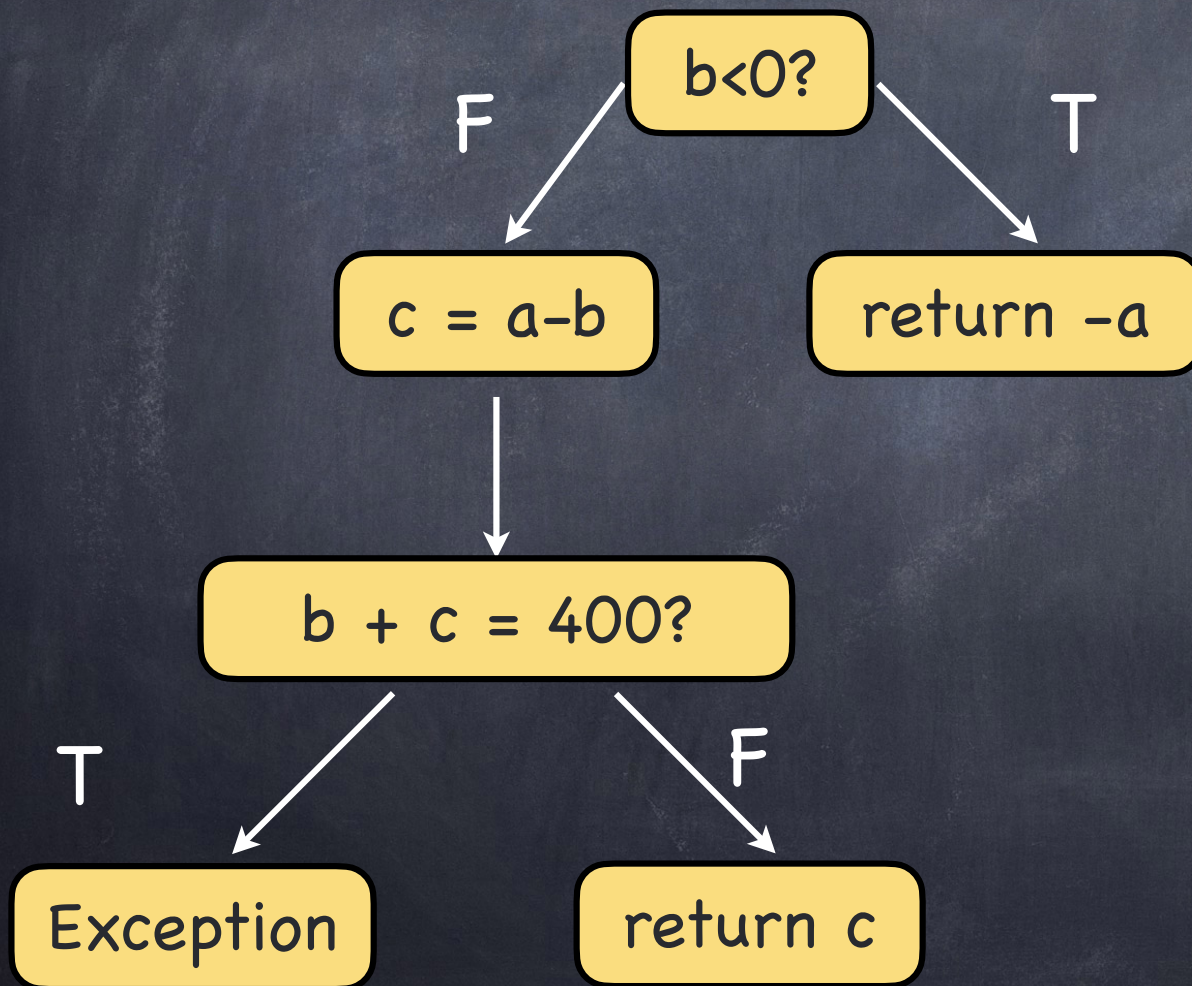

Ejecución Simbólica Dinámica (DSE)

Ejecución Simbólica Dinámica (DSE)



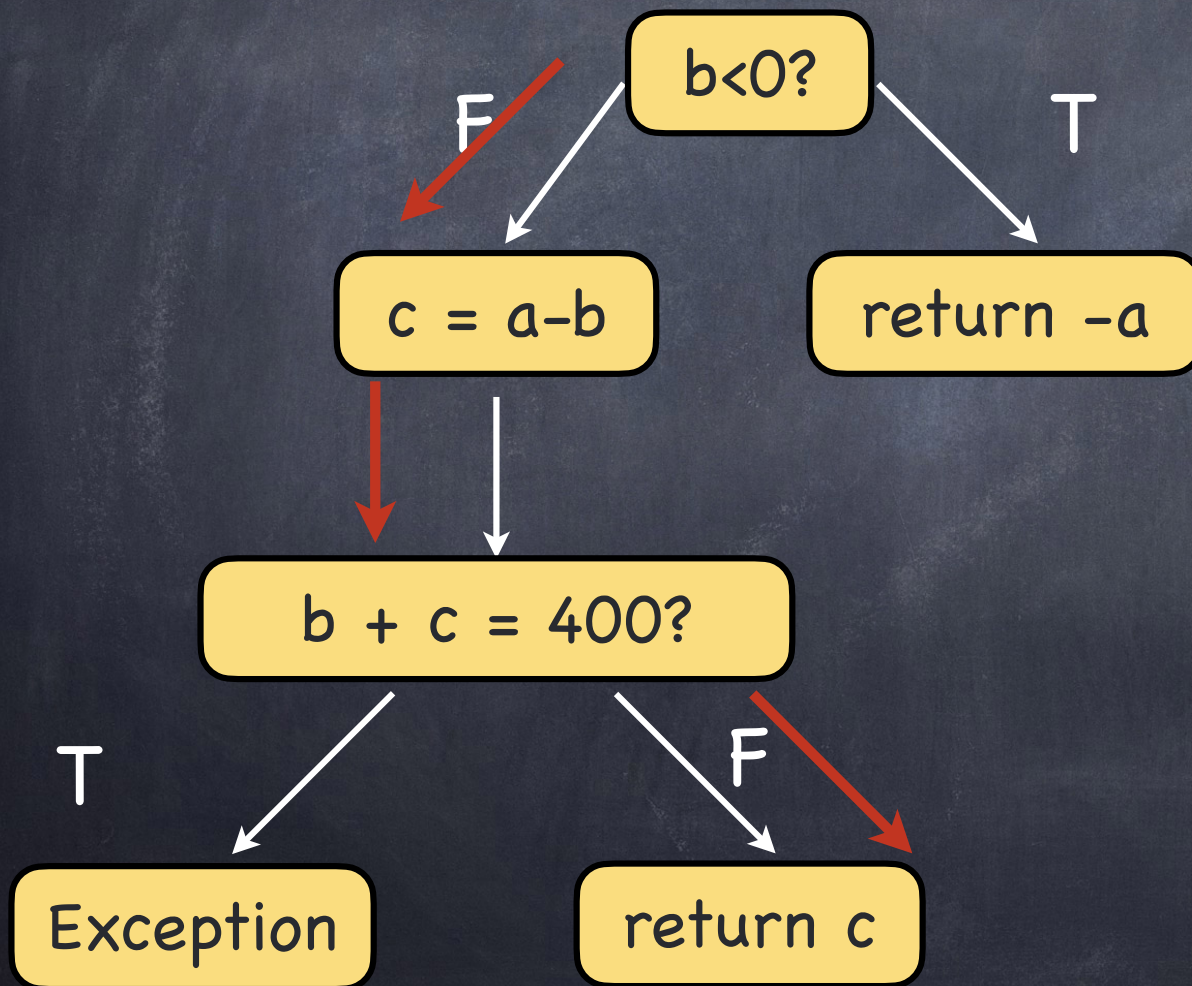
Ejecución Simbólica Dinámica (DSE)

testMe(0,0)



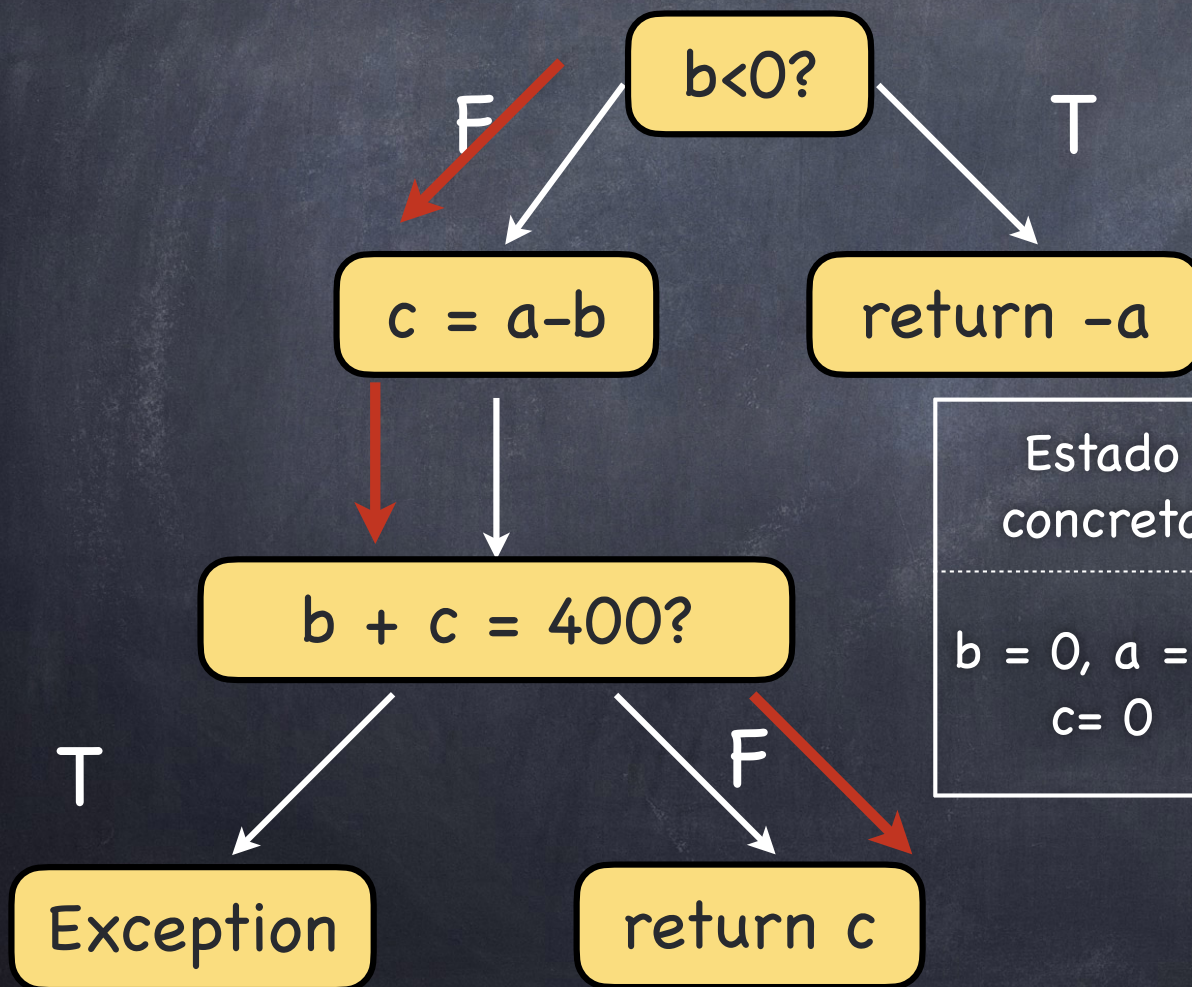
Ejecución Simbólica Dinámica (DSE)

testMe(0,0)



Ejecución Simbólica Dinámica (DSE)

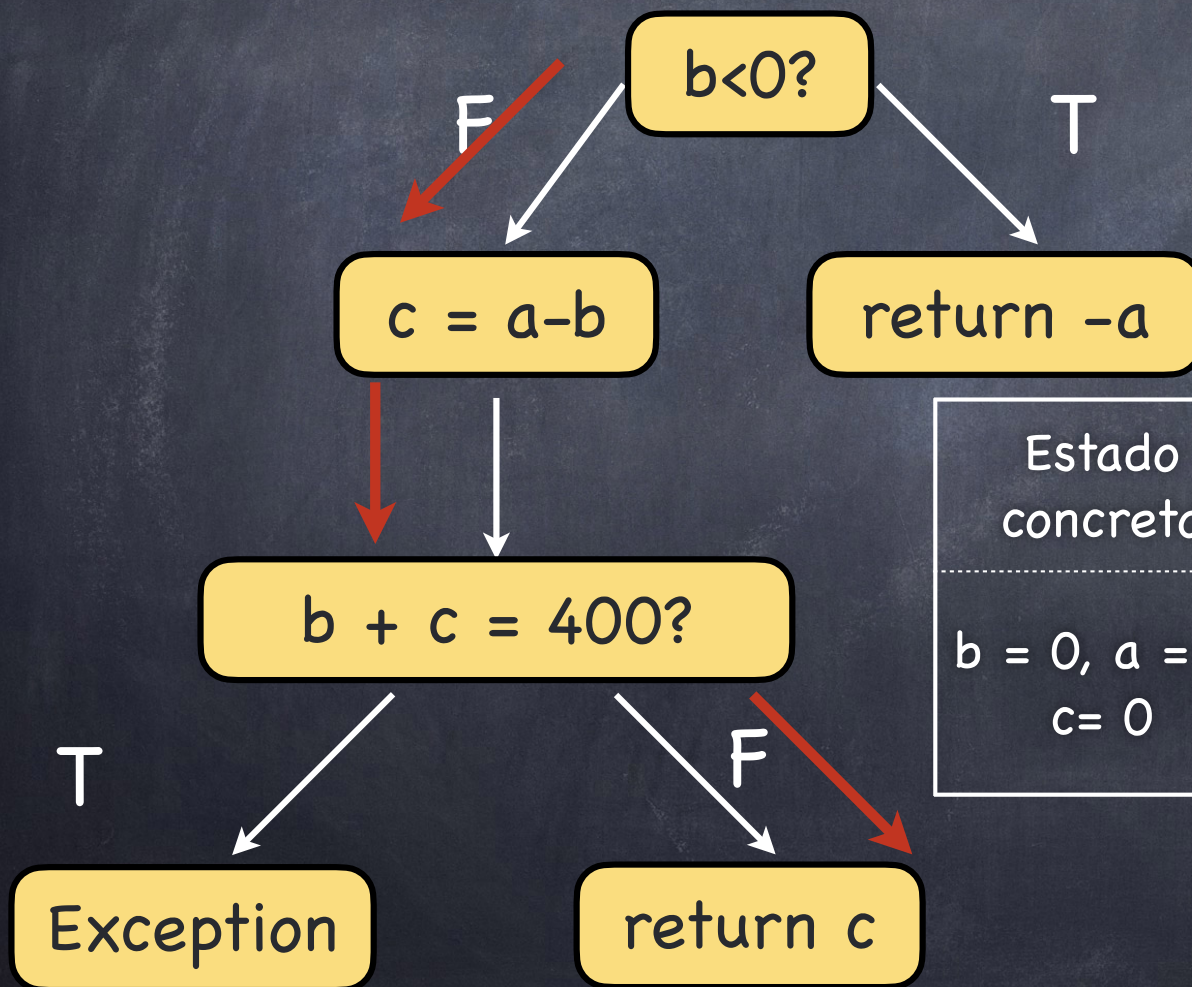
testMe(0,0)



Estado concreto	Estado simbólico	Condición Camino
$b = 0, a = 0, c = 0$	$b = b_0, a = a_0, c = a_0 - b_0$	$b_0 \geq 0 \ \&\& \ b_0 + a_0 - b_0 \neq 400$

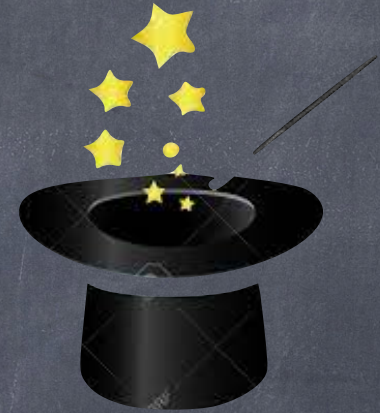
Ejecución Simbólica Dinámica (DSE)

testMe(0,0)

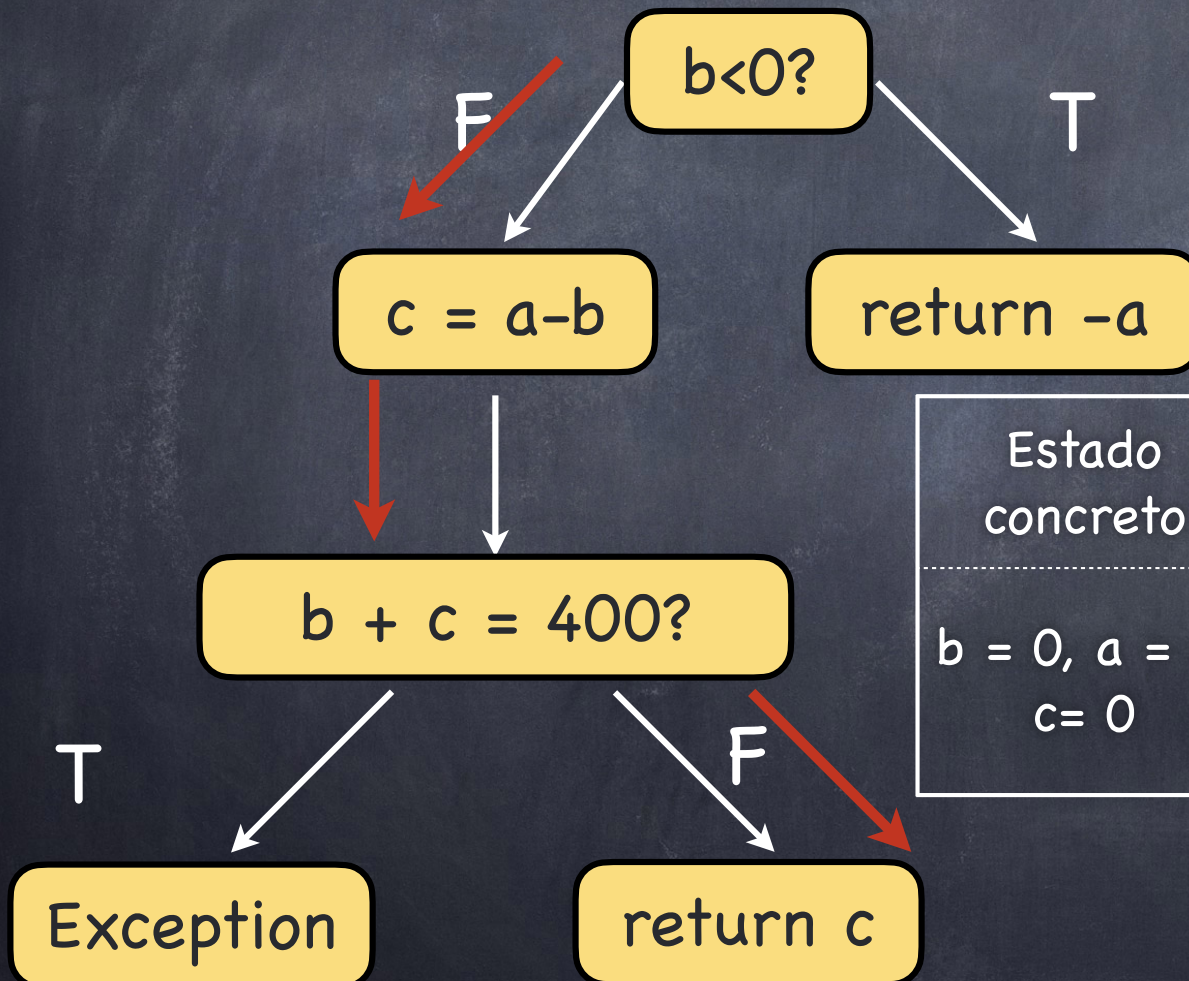


Estado concreto	Estado simbólico	Condición Camino
$b = 0, a = 0, c = 0$	$b = b_0, a = a_0, c = a_0 - b_0$	$b_0 \geq 0 \ \&\& \ a_0 \neq 400$

Ejecución Simbólica Dinámica (DSE)

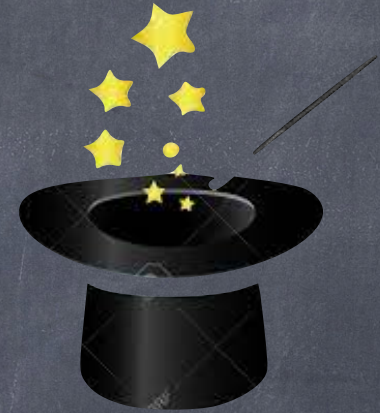


testMe(0,0)

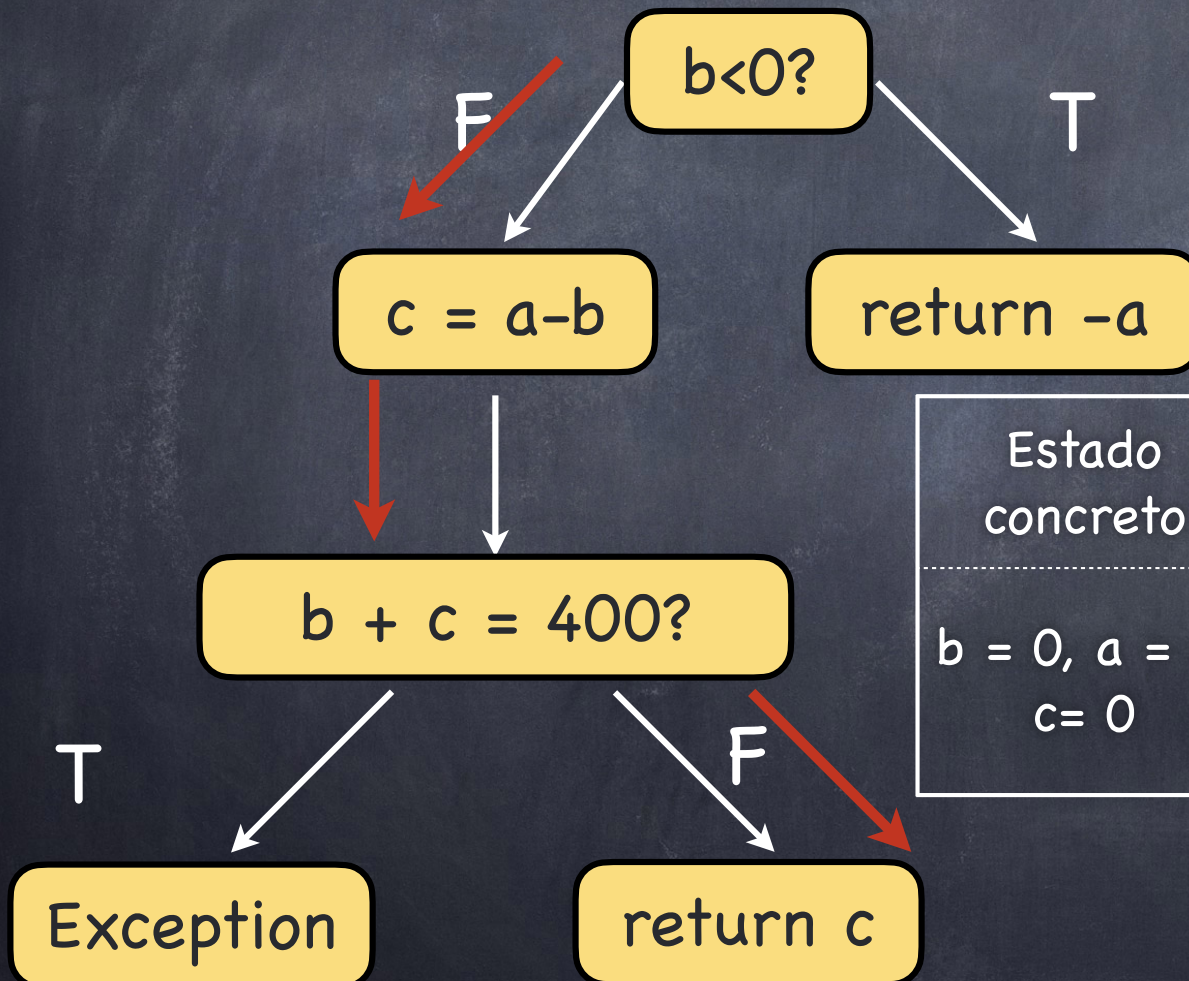


Estado concreto	Estado simbólico	Condición Camino
$b = 0, a = 0, c = 0$	$b = b_0, a = a_0, c = a_0 - b_0$	$b_0 \geq 0 \ \&\& \ a_0 \neq 400$

Ejecución Simbólica Dinámica (DSE)



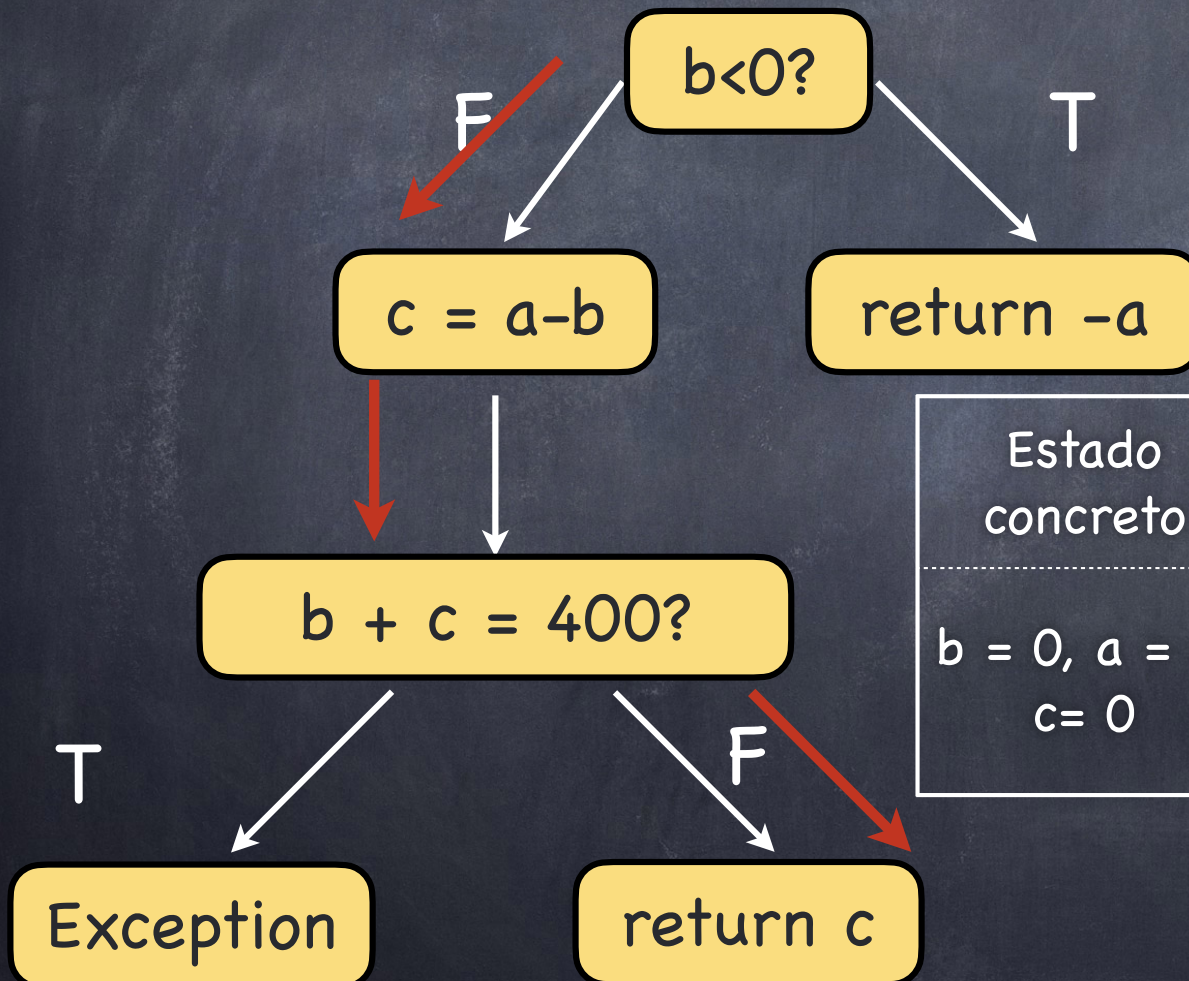
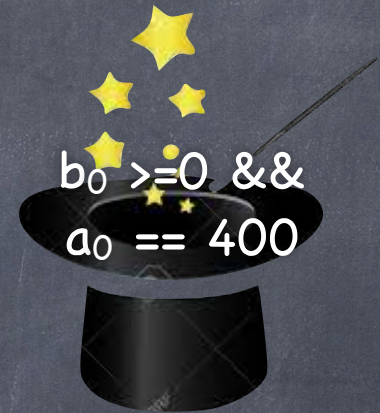
testMe(0,0)



Estado concreto	Estado simbólico	Condición Camino
$b = 0, a = 0, c = 0$	$b = b_0, a = a_0, c = a_0 - b_0$	$b_0 \geq 0 \ \&\& \ a_0 \neq 400$

Ejecución Simbólica Dinámica (DSE)

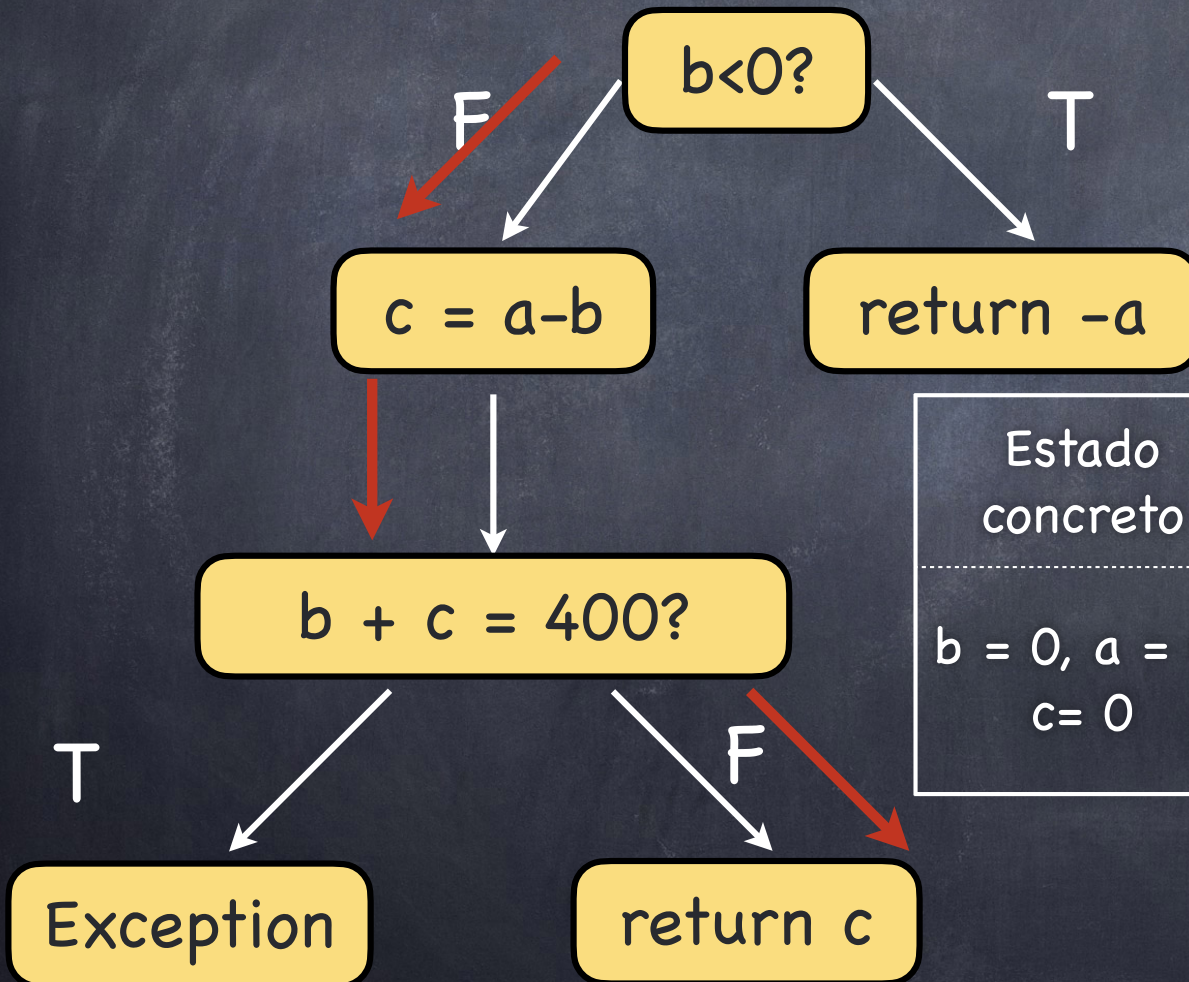
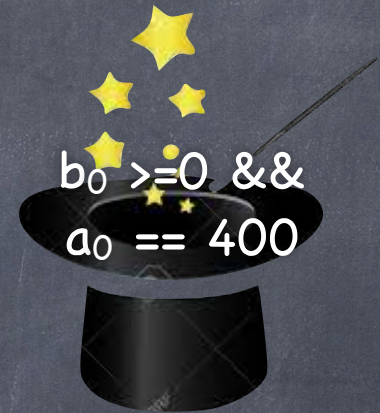
testMe(0,0)



Estado concreto	Estado simbólico	Condición Camino
$b = 0, a = 0, c = 0$	$b = b_0, a = a_0, c = a_0 - b_0$	$b_0 \geq 0 \ \&\& \ a_0 \neq 400$

Ejecución Simbólica Dinámica (DSE)

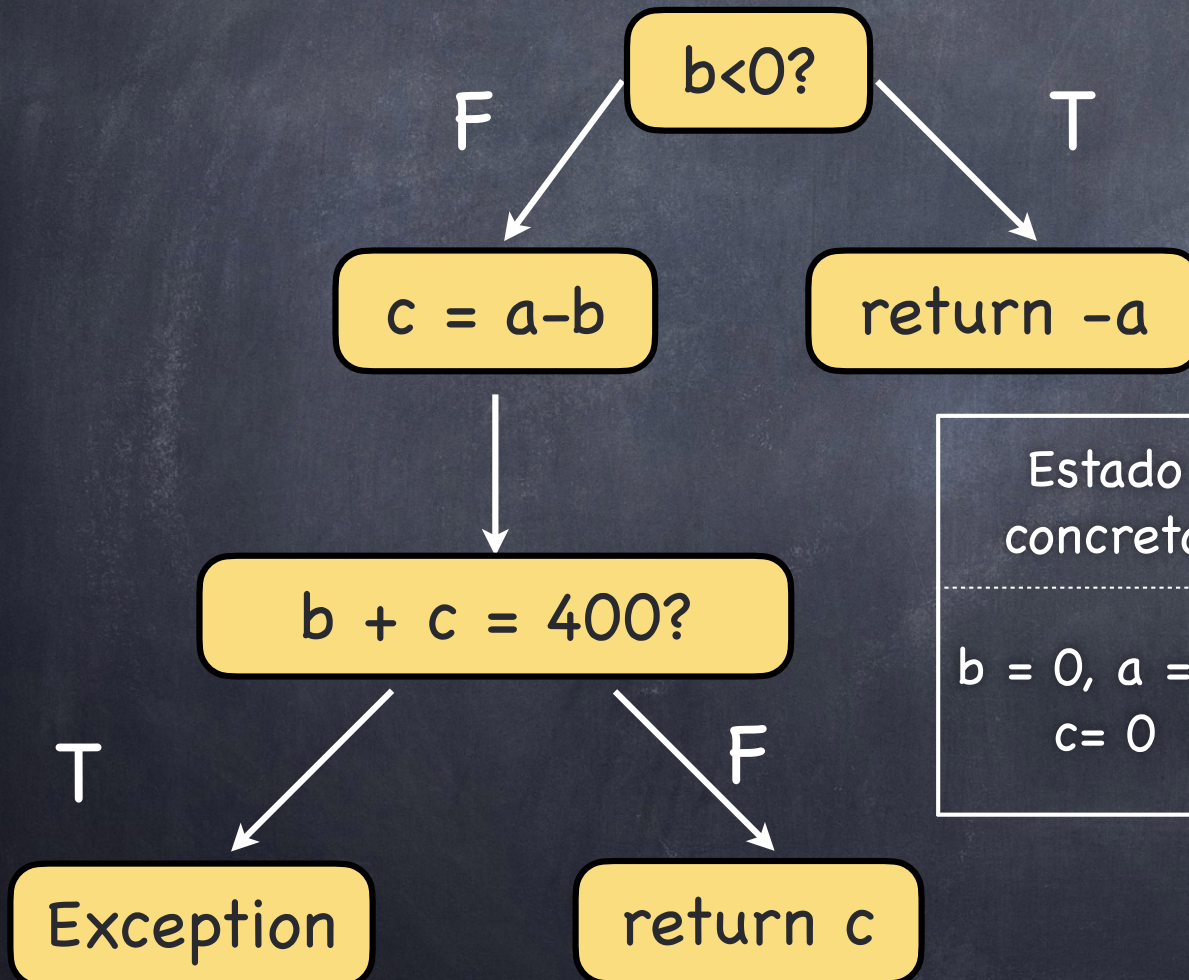
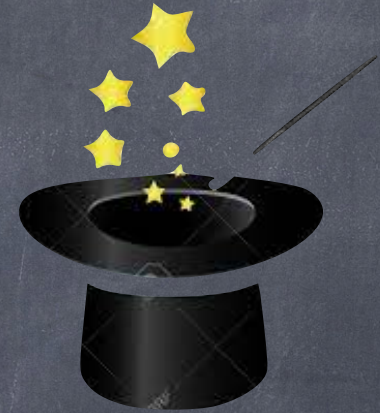
testMe(0,0)



Estado concreto	Estado simbólico	Condición Camino
$b = 0, a = 0, c = 0$	$b = b_0, a = a_0, c = a_0 - b_0$	$b_0 \geq 0 \ \&\& \ a_0 \neq 400$

Ejecución Simbólica Dinámica (DSE)

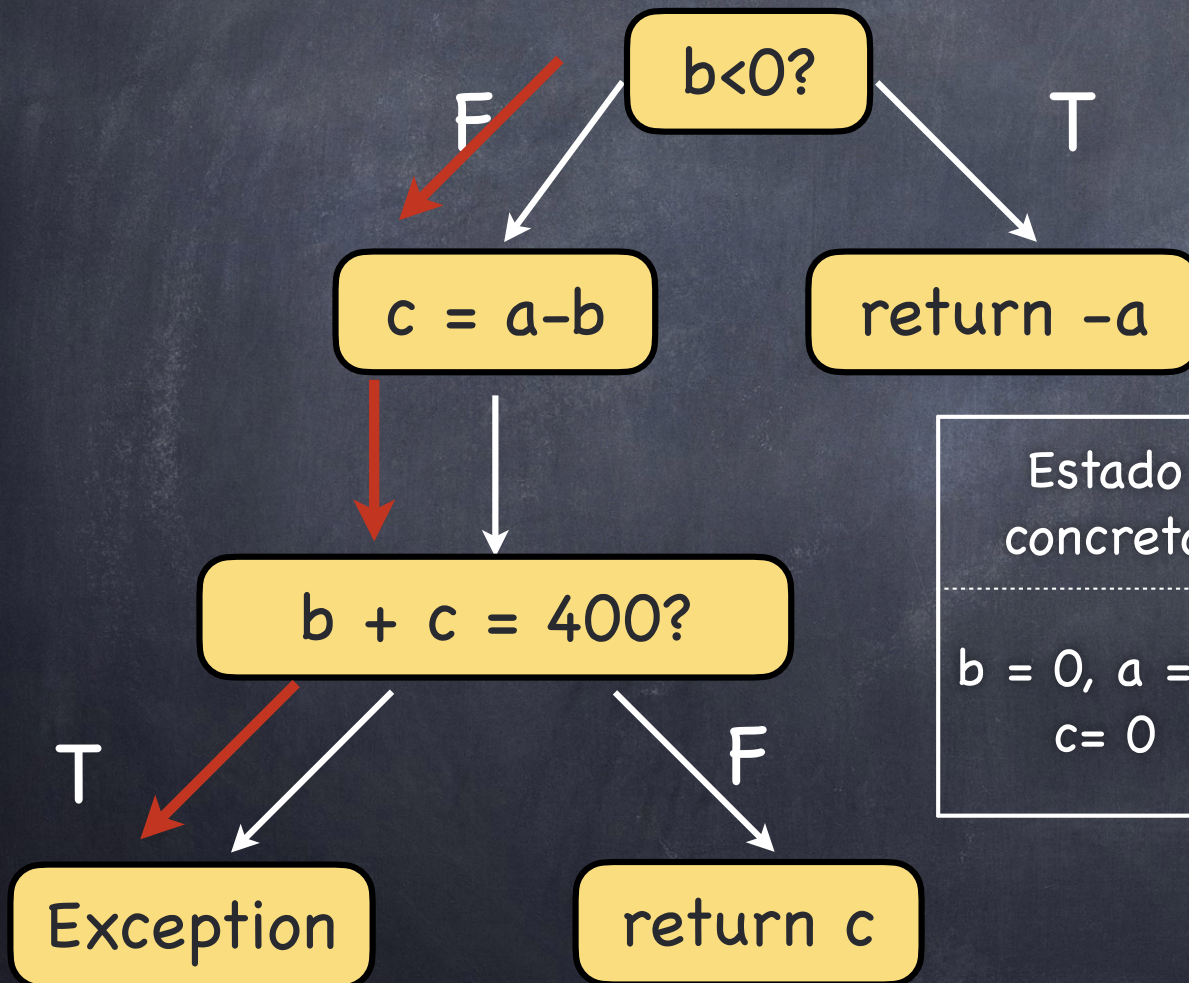
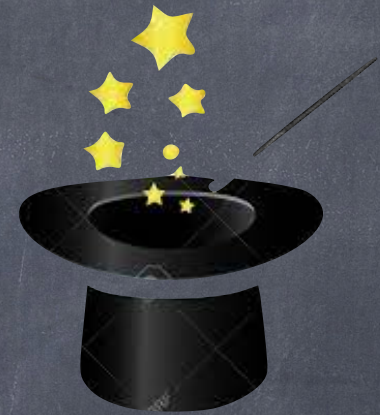
testMe(400, 0)



Estado concreto	Estado simbólico	Condición Camino
$b = 0, a = 0, c = 0$	$b = b_0, a = a_0, c = a_0 - b_0$	$b_0 \geq 0 \ \&\& \ a_0 \neq 400$

Ejecución Simbólica Dinámica (DSE)

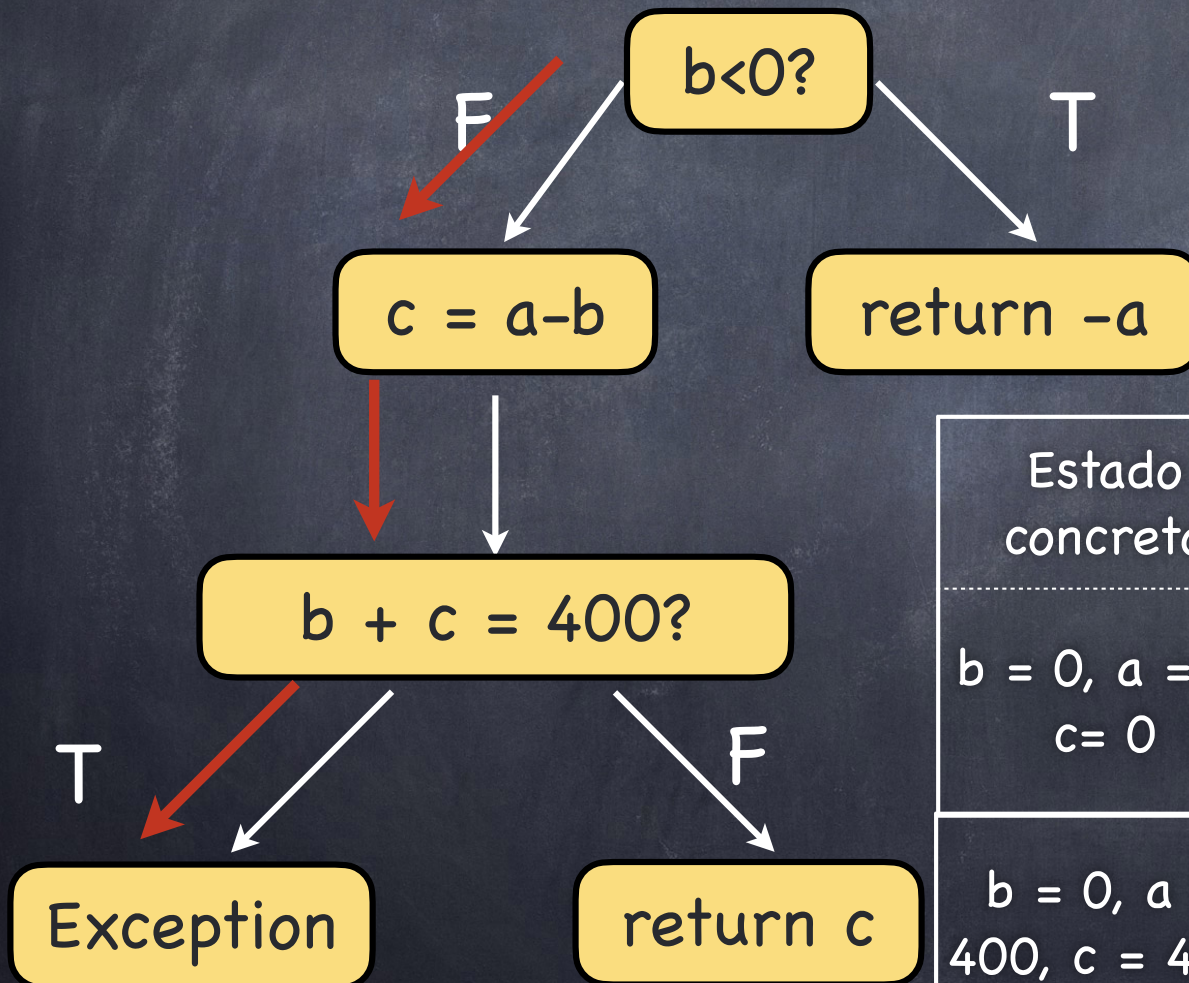
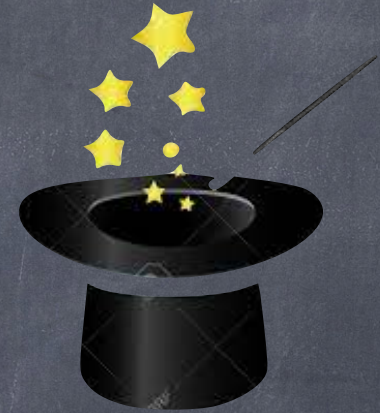
testMe(400, 0)



Estado concreto	Estado simbólico	Condición Camino
$b = 0, a = 0, c = 0$	$b = b_0, a = a_0, c = a_0 - b_0$	$b_0 \geq 0 \ \&\& \ a_0 \neq 400$

Ejecución Simbólica Dinámica (DSE)

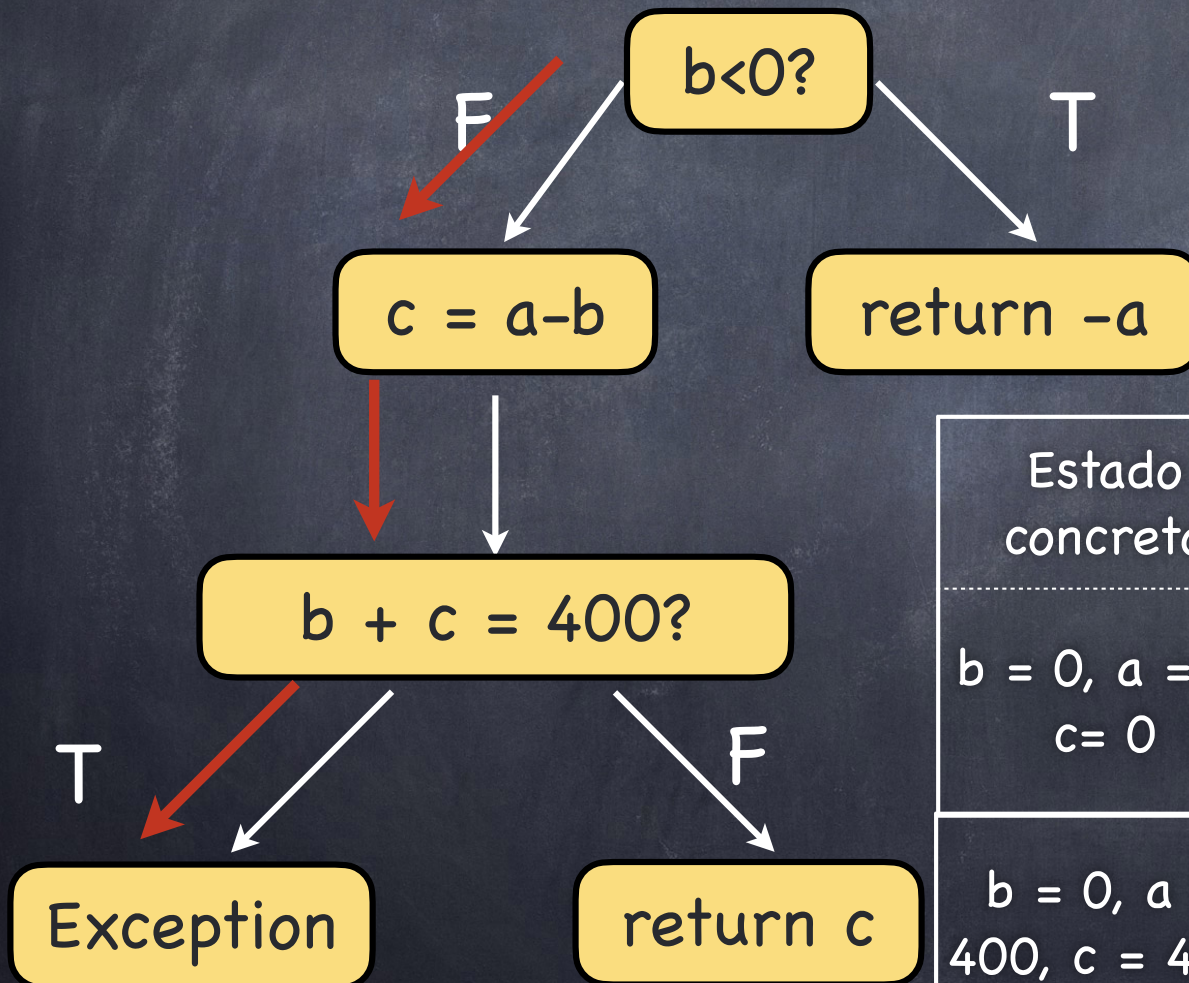
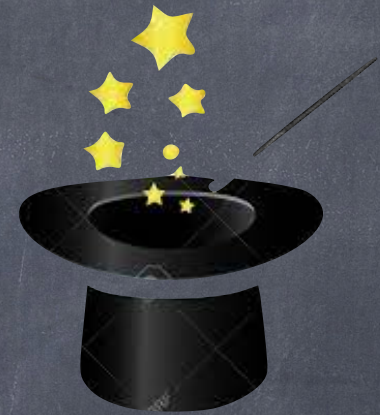
testMe(400, 0)



Estado concreto	Estado simbólico	Condición Camino
$b = 0, a = 0, c = 0$	$b = b_0, a = a_0, c = a_0 - b_0$	$b_0 \geq 0 \ \&\& \ a_0 \neq 400$
$b = 0, a = 400, c = 400$	$b = b_0, a = a_0, c = a_0 - b_0$	$b_0 \geq 0 \ \&\& \ a_0 == 400$

Ejecución Simbólica Dinámica (DSE)

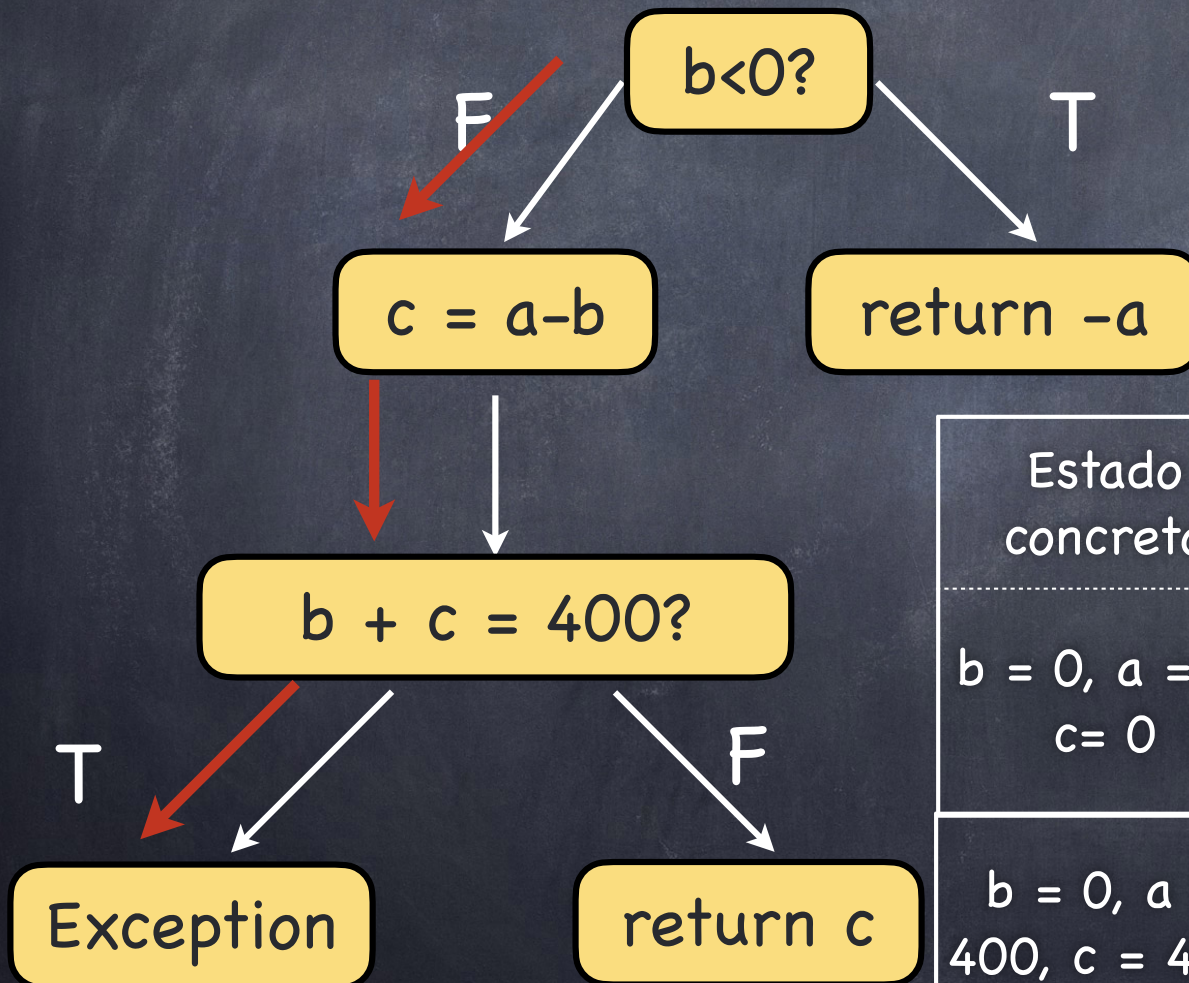
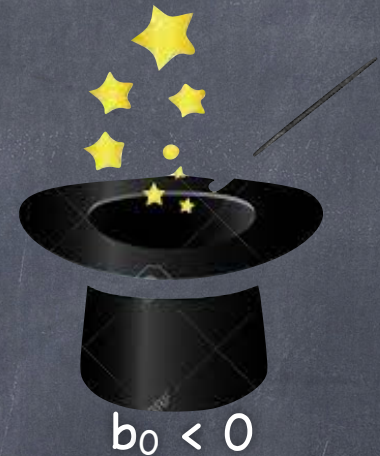
testMe(400, 0)



Estado concreto	Estado simbólico	Condición Camino
$b = 0, a = 0, c = 0$	$b = b_0, a = a_0, c = a_0 - b_0$	$b_0 \geq 0 \ \&\& \ a_0 \neq 400$
$b = 0, a = 400, c = 400$	$b = b_0, a = a_0, c = a_0 - b_0$	$b_0 \geq 0 \ \&\& \ a_0 == 400$

Ejecución Simbólica Dinámica (DSE)

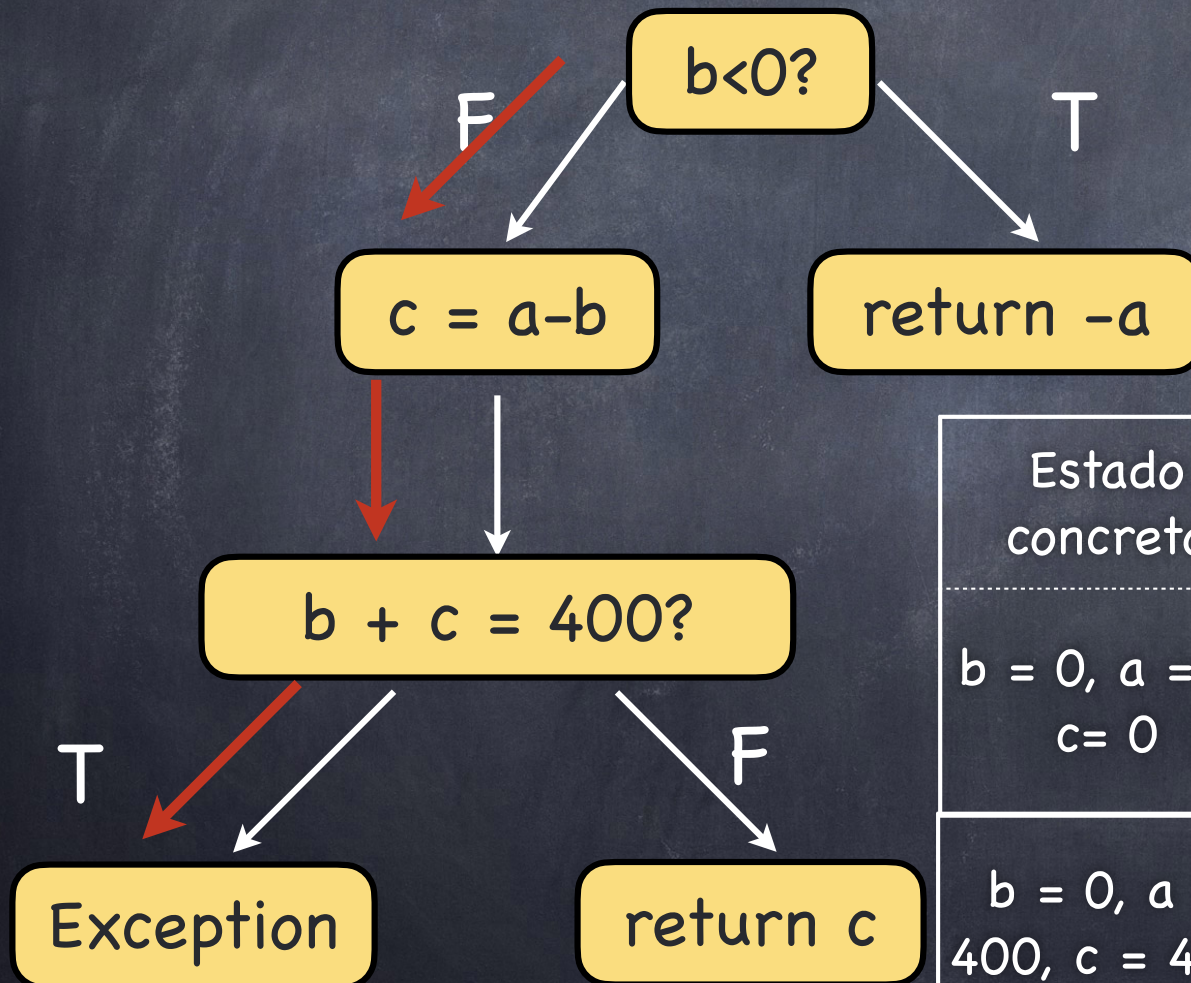
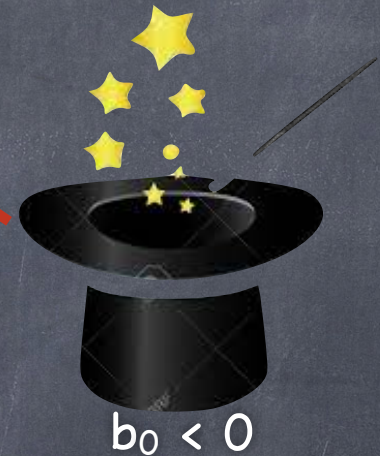
testMe(400, 0)



Estado concreto	Estado simbólico	Condición Camino
$b = 0, a = 0, c = 0$	$b = b_0, a = a_0, c = a_0 - b_0$	$b_0 \geq 0 \ \&\& \ a_0 \neq 400$
$b = 0, a = 400, c = 400$	$b = b_0, a = a_0, c = a_0 - b_0$	$b_0 \geq 0 \ \&\& \ a_0 == 400$

Ejecución Simbólica Dinámica (DSE)

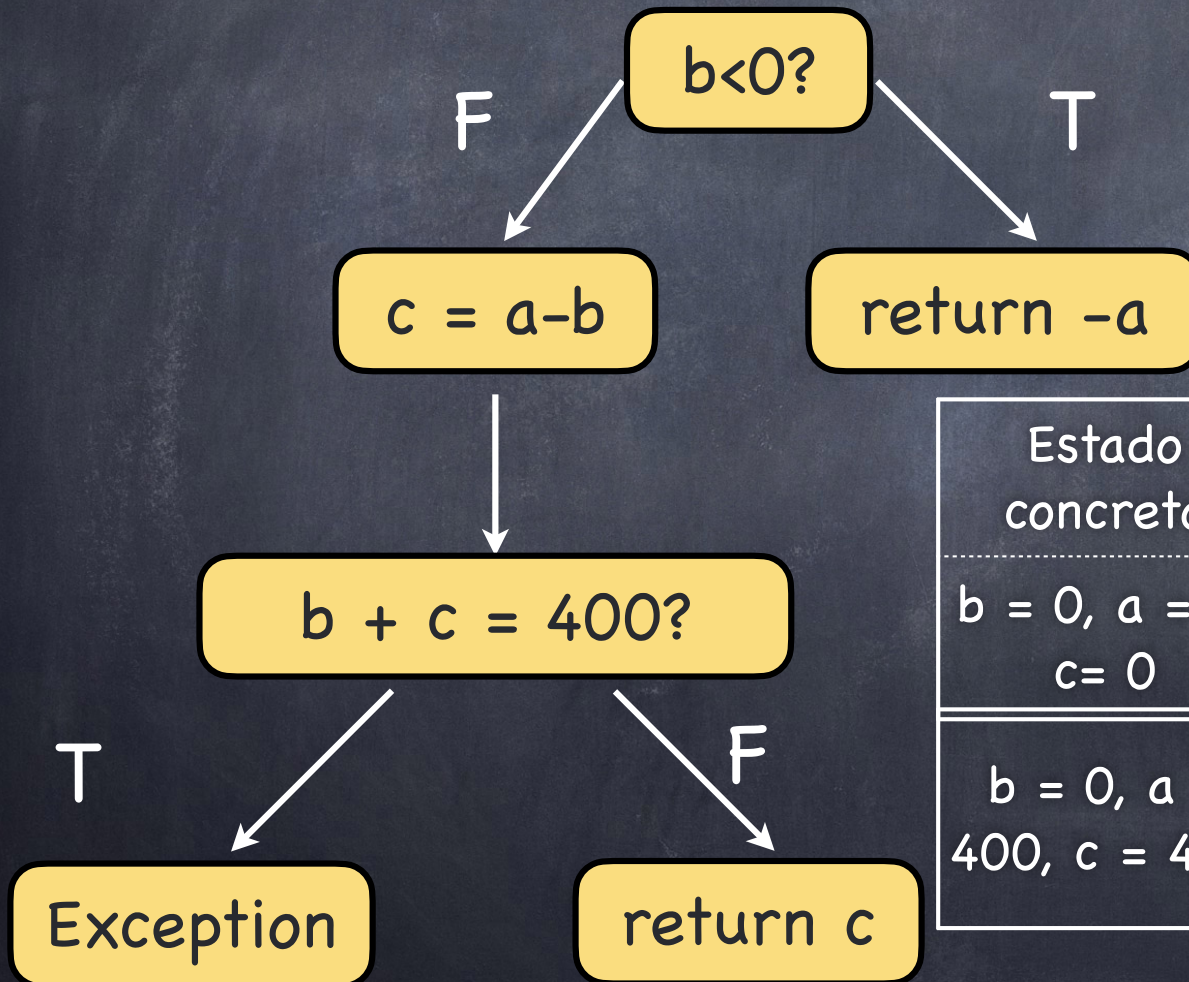
testMe(400, 0)



Estado concreto	Estado simbólico	Condición Camino
$b = 0, a = 0, c = 0$	$b = b_0, a = a_0, c = a_0 - b_0$	$b_0 \geq 0 \ \&\& \ a_0 \neq 400$
$b = 0, a = 400, c = 400$	$b = b_0, a = a_0, c = a_0 - b_0$	$b_0 \geq 0 \ \&\& \ a_0 == 400$

Ejecución Simbólica Dinámica (DSE)

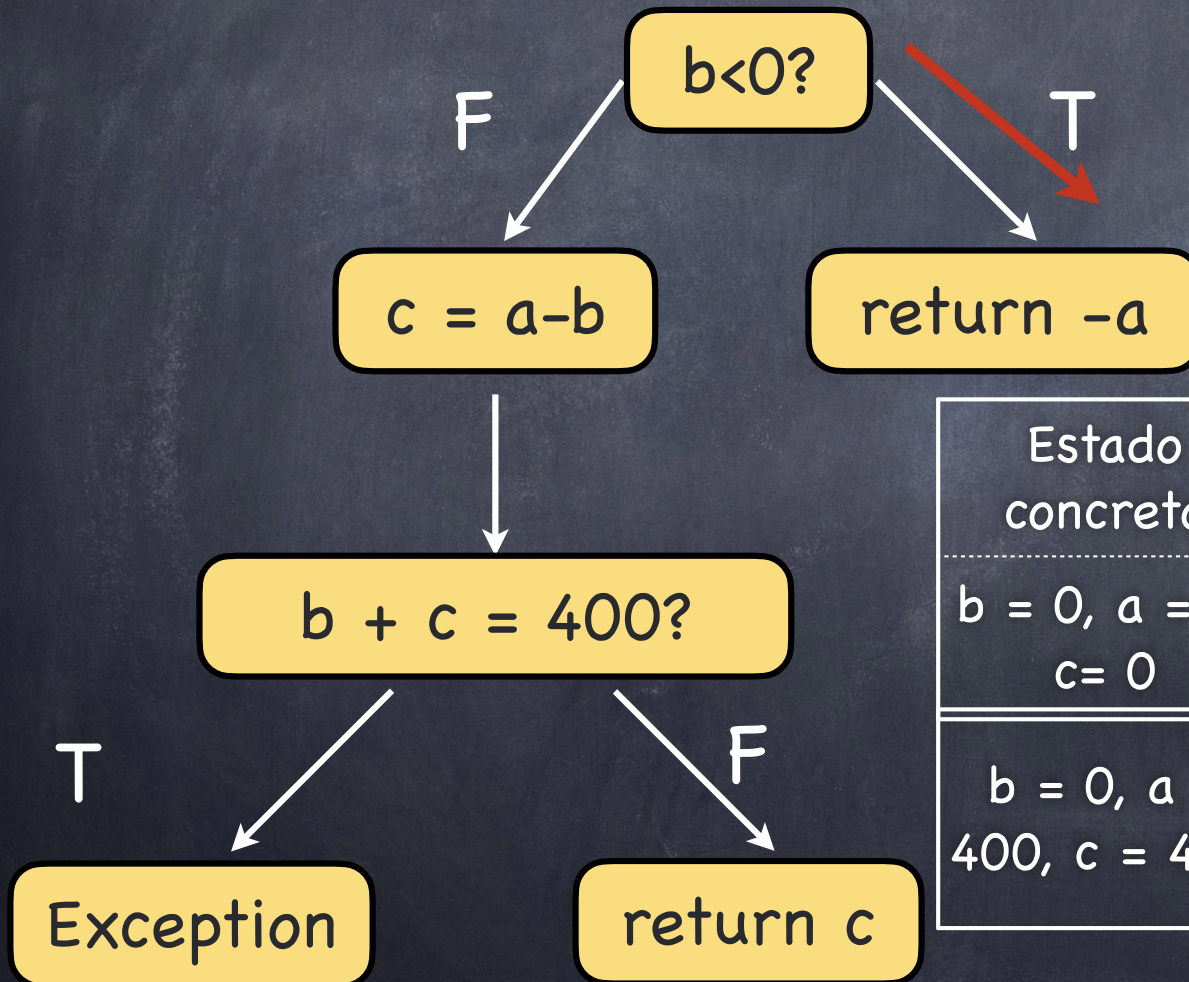
testMe(400, -1)



Estado concreto	Estado simbólico	Condición Camino
$b = 0, a = 0, c = 0$	$b = b_0, a = a_0, c = a_0 - b_0$	$b_0 \geq 0 \ \&\& \ a_0 \neq 400$
$b = 0, a = 400, c = 400$	$b = b_0, a = a_0, c = a_0 - b_0$	$b_0 \geq 0 \ \&\& \ a_0 == 400$

Ejecución Simbólica Dinámica (DSE)

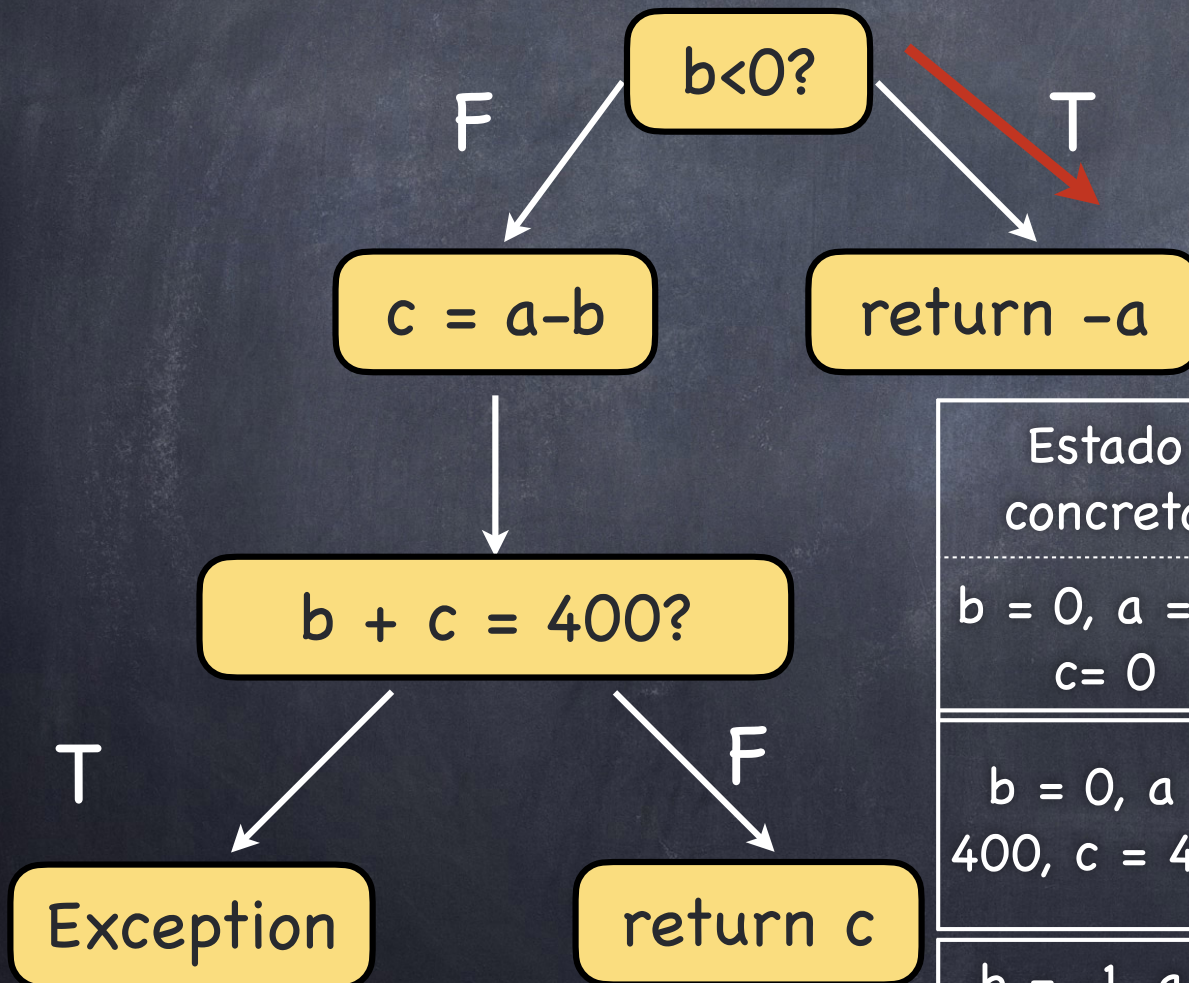
testMe(400, -1)



Estado concreto	Estado simbólico	Condición Camino
$b = 0, a = 0, c = 0$	$b = b_0, a = a_0, c = a_0 - b_0$	$b_0 \geq 0 \ \&\& \ a_0 \neq 400$
$b = 0, a = 400, c = 400$	$b = b_0, a = a_0, c = a_0 - b_0$	$b_0 \geq 0 \ \&\& \ a_0 == 400$

Ejecución Simbólica Dinámica (DSE)

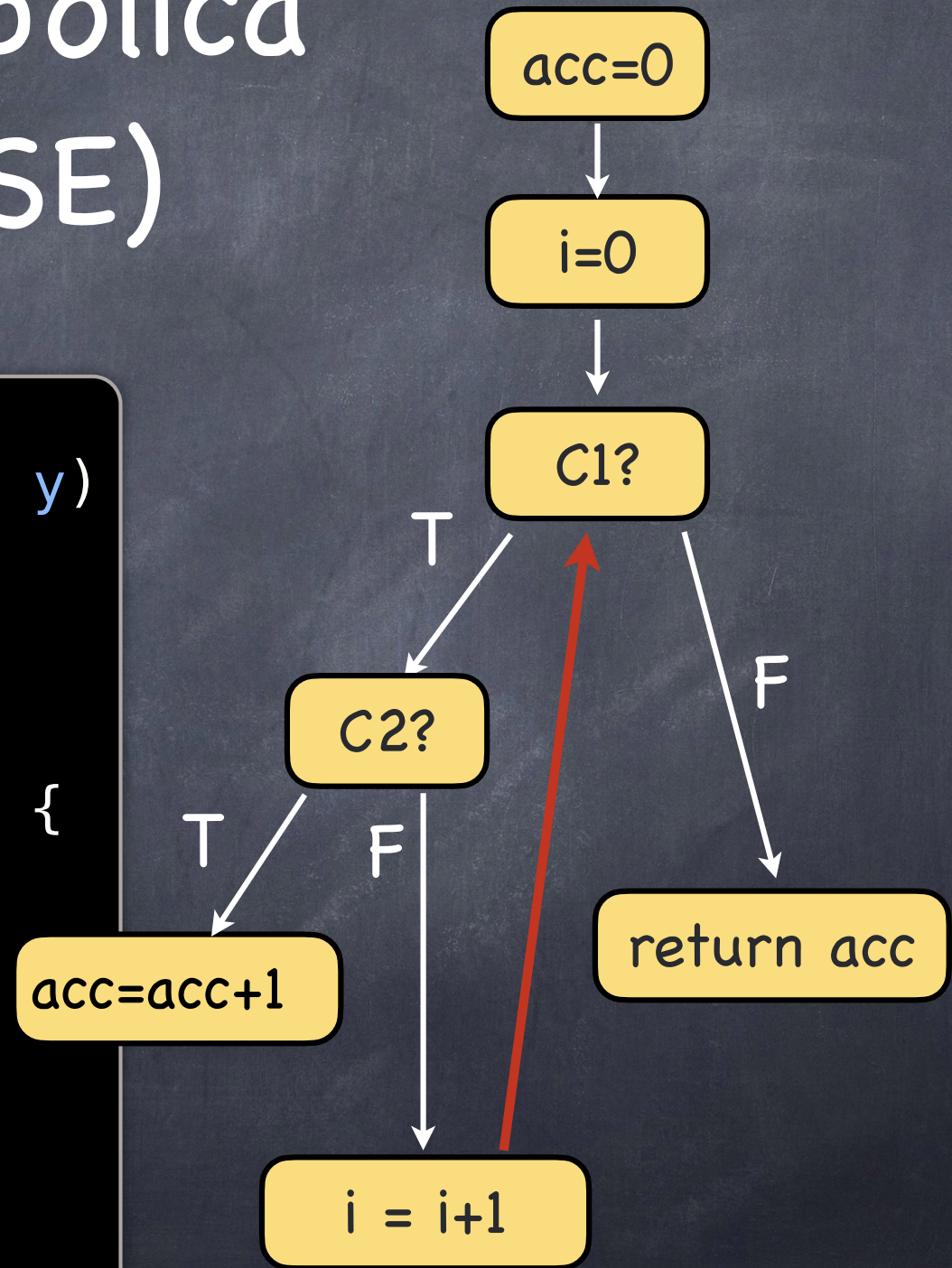
testMe(400, -1)



Estado concreto	Estado simbólico	Condición Camino
$b = 0, a = 0, c = 0$	$b = b_0, a = a_0, c = a_0 - b_0$	$b_0 \geq 0 \ \&\& \ a_0 \neq 400$
$b = 0, a = 400, c = 400$	$b = b_0, a = a_0, c = a_0 - b_0$	$b_0 \geq 0 \ \&\& \ a_0 == 400$
$b = -1, a = 400$	$b = b_0, a = a_0$	$b_0 < 0$

Ejecución Simbólica Dinámica (DSE)

```
public int testMe(int x, int y)
{
    int acc=0;
    int i=0;
    while (i<y) {
        if ((x % (i+1)) == 0 ) {
            acc=acc+1;
        }
        i=i+1;
    }
    return acc;
}
```



Loops Unroll

Dado un programa, su versión desenrollada (unrolled) corresponde a la versión del programa donde cada ciclo se ha desenrollado una cantidad dada de veces

Este se utiliza para poder aplicar DSE a programas con ciclos, que son potencialmente infinitos.

Loops Unroll

```
public int testMe(int x, int y)
{
    int acc=0;
    int i=0;
    while (c1) {
        if (c2){
            acc=acc+1;
        }
        i=i+1;
    }
    return acc;
}
```

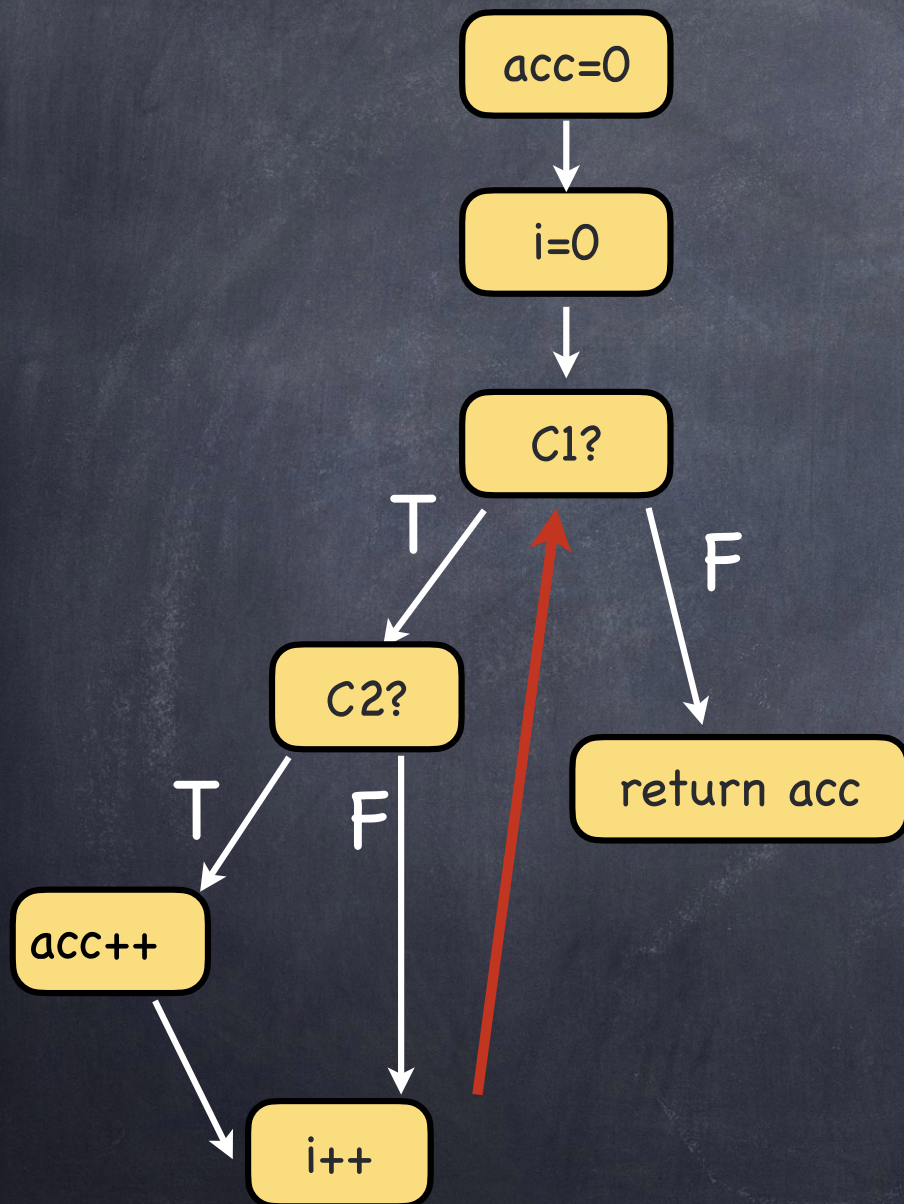

Loops Unroll

```
public int testMe(int x,
{
    int acc=0;
    int i=0;
    while (c1) {
        if (c2){
            acc=acc+1;
        }
        i=i+1;
    }
    return acc;
}
```

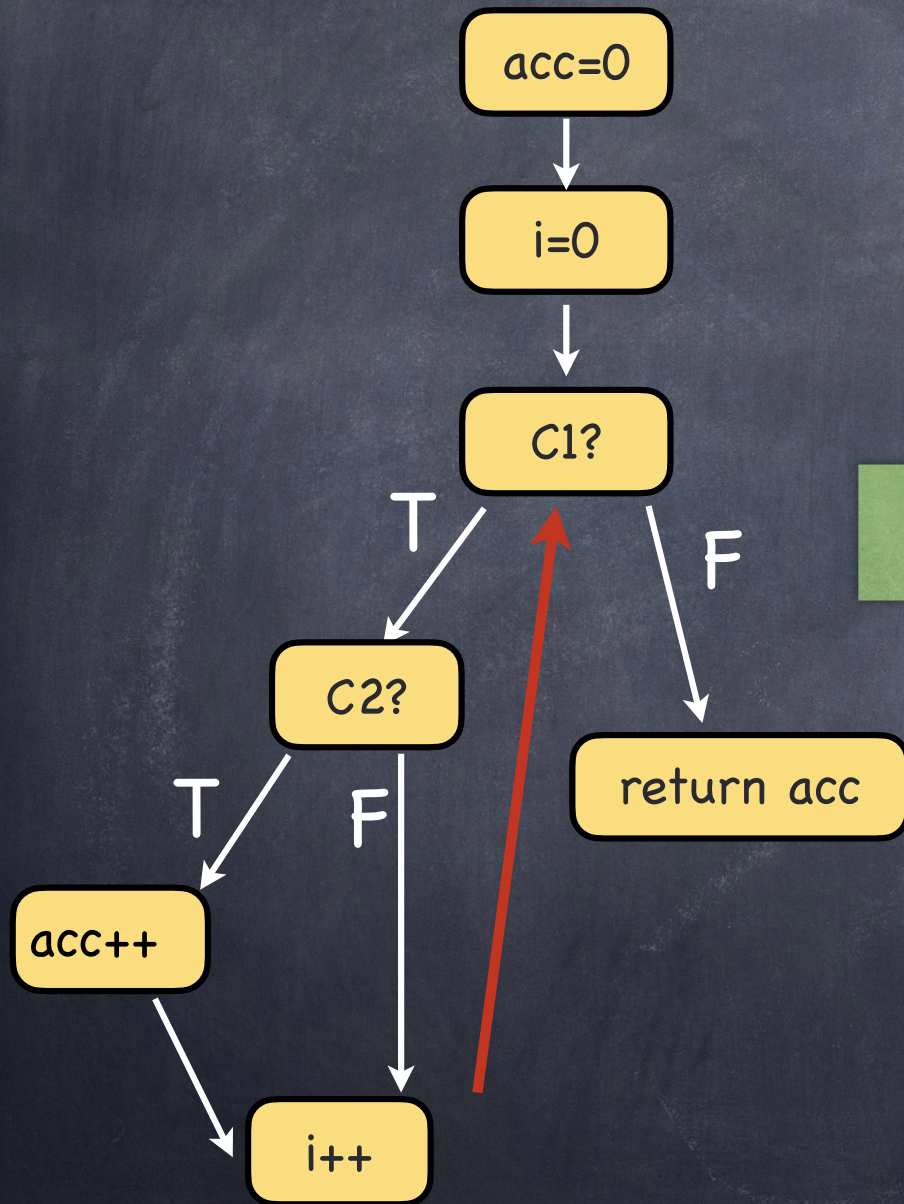
2 unrolls

```
int testMe(int x, int k) {
    int acc=0;
    int i=0;
    if (i<k) { //c1_0
        if ((x%(i+1))==0) { //c2_0
            acc=acc+1;
        }
        i=i+1;
    }
    if (i<k) { //c1_1
        if ((x%(i+1))==0) { //c2_1
            acc=acc+1;
        }
        i=i+1;
    }
    if (i<k) { //c1_1
        ASSUME FALSE;
    }
    }
    return acc;
}
```

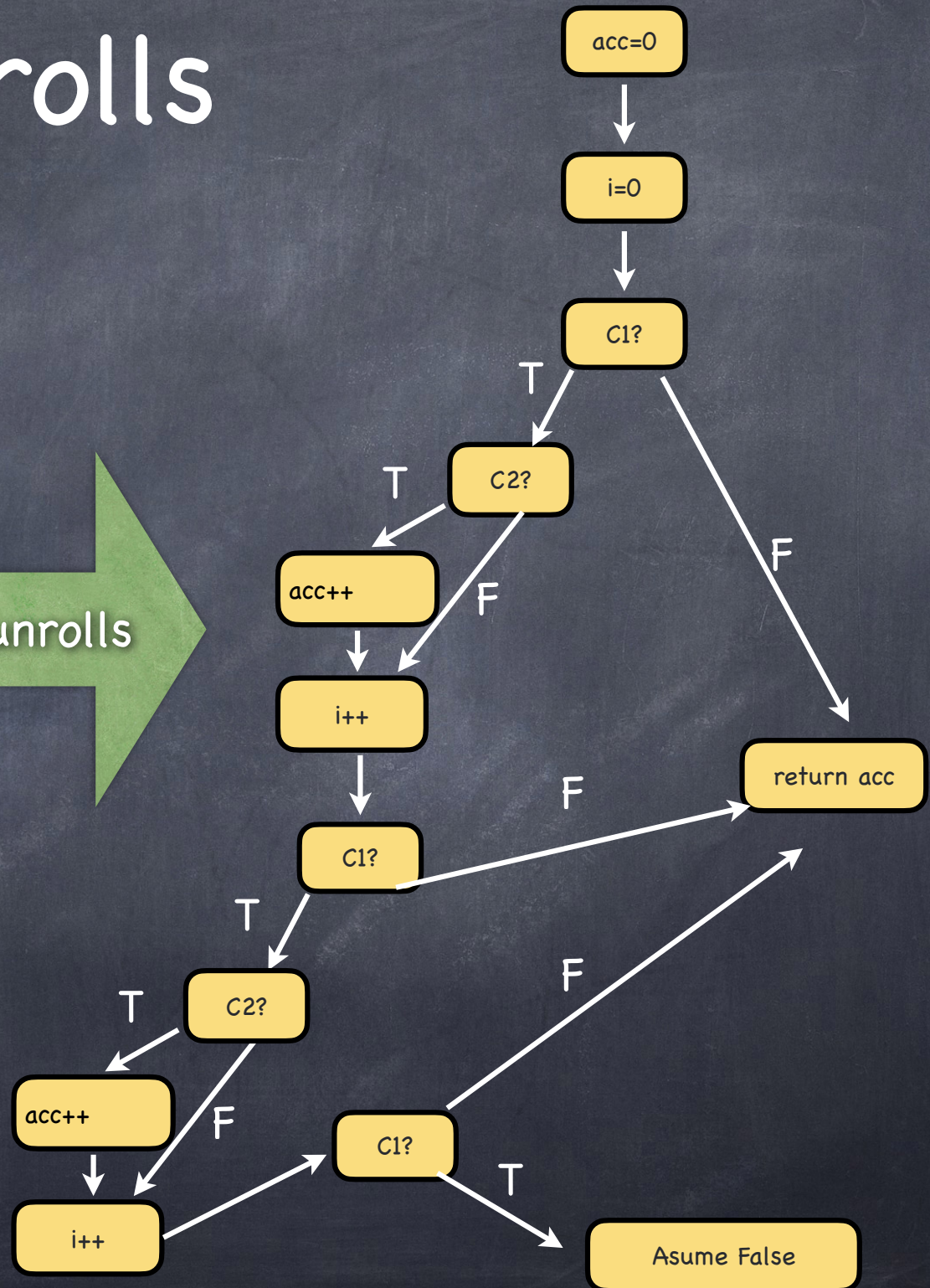

Loops Unrolls



Loops Unrolls



2 unrolls



Ejecución Simbólica Dinámica (DSE)

```
public static int foo(int v) {  
    return someValueFrom(v);  
}  
  
public static void test_me(int x, int y) {  
    int z = foo(y);  
  
    if (z == x) {  
        if (x > y+10) {  
            // ERROR;  
        }  
    }  
}
```

Estado concreto

Estado simbólico

Condición Camino

Ejecución Simbólica Dinámica (DSE)

```
public static int foo(int v) {  
    return someValueFrom(v);  
}
```

test_me(15,7)

```
public static void test_me(int x, int y) {  
    int z = foo(y);  
  
    if (z == x) {  
        if (x > y+10) {  
            // ERROR;  
        }  
    }  
}
```

Estado concreto

Estado simbólico

Condición Camino

Ejecución Simbólica Dinámica (DSE)

```
public static int foo(int v) {  
    return someValueFrom(v);  
}
```

test_me(15,7)

```
public static void test_me(int x, int y) {  
    int z = foo(y);  
  
    if (z == x) {  
        if (x > y+10) {  
            // ERROR;  
        }  
    }  
}
```

Estado concreto	Estado simbólico	Condición Camino
x = 15, y = 7, z = 5467		

Ejecución Simbólica Dinámica (DSE)

```
public static int foo(int v) {  
    return someValueFrom(v);  
}
```

test_me(15,7)

```
public static void test_me(int x, int y) {  
    int z = foo(y);  
  
    if (z == x) {  
        if (x > y+10) {  
            // ERROR;  
        }  
    }  
}
```

Estado concreto	Estado simbólico	Condición Camino
$x = 15, y = 7, z = 5467$	$x = x0, y = y0,$ $z0 = \text{someValueFrom}(y0)$	

Ejecución Simbólica Dinámica (DSE)

```
public static int foo(int v) {  
    return someValueFrom(v);  
}
```

test_me(15,7)

```
public static void test_me(int x, int y) {  
    int z = foo(y);  
  
    if (z == x) {  
        if (x > y+10) {  
            // ERROR;  
        }  
    }  
}
```

Estado concreto	Estado simbólico	Condición Camino
$x = 15, y = 7, z = 5467$	$x = x0, y = y0,$ $z0 = \text{someValueFrom}(y0)$	$X0 \neq \text{someValueFrom}(y0)$

Ejecución Simbólica Dinámica (DSE)

```
public static int foo(int v) {  
    return someValueFrom(v);  
}
```

```
public static void test_me(int x, int y) {  
    int z = foo(y);  
  
    if (z == x) {  
        if (x > y+10) {  
            // ERROR;  
        }  
    }  
}
```

test_me(15,7)

X0 == someValueFrom(y0)



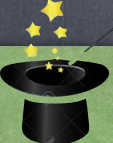
Estado concreto	Estado simbólico	Condición Camino
x = 15, y = 7, z = 5467	x = x0, y = y0, z0 = someValueFrom(y0)	X0 != someValueFrom(y0)

Ejecución Simbólica Dinámica (DSE)

```
public static int foo(int v) {  
    return someValueFrom(v);  
}  
  
public static void test_me(int x, int y) {  
    int z = foo(y);  
  
    if (z == x) {  
        if (x > y+10) {  
            // ERROR;  
        }  
    }  
}
```

test_me(15,7)

Usamos el valor
concreto de y_0 (7)


 $X0 == \text{someValueFrom}(y0)$

Estado concreto	Estado simbólico	Condición Camino
$x = 15, y = 7, z = 5467$	$x = x0, y = y0,$ $z0 = \text{someValueFrom}(y0)$	$X0 != \text{someValueFrom}(y0)$

Ejecución Simbólica Dinámica (DSE)

```
public static int foo(int v) {  
    return someValueFrom(v);  
}  
  
public static void test_me(int x, int y) {  
    int z = foo(y);  
  
    if (z == x) {  
        if (x > y+10) {  
            // ERROR;  
        }  
    }  
}
```

Estado concreto

Estado simbólico

Condición Camino

Ejecución Simbólica Dinámica (DSE)

```
public static int foo(int v) {  
    return someValueFrom(v);  
}
```

```
public static void test_me(int x, int y) {  
    int z = foo(y);  
  
    if (z == x) {  
        if (x > y+10) {  
            // ERROR;  
        }  
    }  
}
```

test_me(5467,7)

Estado concreto

Estado simbólico

Condición Camino

Ejecución Simbólica Dinámica (DSE)

```
public static int foo(int v) {  
    return someValueFrom(v);  
}
```

```
public static void test_me(int x, int y) {  
    int z = foo(y);  
  
    if (z == x) {  
        if (x > y+10) {  
            // ERROR;  
        }  
    }  
}
```

test_me(5467,7)

Estado concreto	Estado simbólico	Condición Camino
x = 5467, y = 7, z = 5467		

Ejecución Simbólica Dinámica (DSE)

```
public static int foo(int v) {  
    return someValueFrom(v);  
}
```

```
public static void test_me(int x, int y) {  
    int z = foo(y);  
  
    if (z == x) {  
        if (x > y+10) {  
            // ERROR;  
        }  
    }  
}
```

test_me(5467,7)

Estado concreto	Estado simbólico	Condición Camino
$x = 5467, y = 7, z = 5467$	$x = x0, y = y0,$ $z0 = \text{someValueFrom}(y0)$	

Ejecución Simbólica Dinámica (DSE)

```
public static int foo(int v) {  
    return someValueFrom(v);  
}
```

```
public static void test_me(int x, int y) {  
    int z = foo(y);  
  
    if (z == x) {  
        if (x > y+10) {  
            // ERROR;  
        }  
    }  
}
```

test_me(5467,7)

Estado concreto	Estado simbólico	Condición Camino
$x = 5467, y = 7, z = 5467$	$x = x0, y = y0,$ $z0 = \text{someValueFrom}(y0)$	$X0 == \text{someValueFrom}(y0)$ && $x0 > y0 + 10$

Ejecución Simbólica Dinámica (DSE)

```
public static int foo(int v) {  
    return someValueFrom(v);  
}
```

```
public static void test_me(int x, int y) {  
    int z = foo(y);
```

```
    if (z == x) {  
        if (x > y+10) {  
            // ERROR;  
        }  
    }
```

```
}
```

test_me(5467,7)

CRASH!

Estado concreto	Estado simbólico	Condición Camino
$x = 5467, y = 7, z = 5467$	$x = x0, y = y0,$ $z0 = \text{someValueFrom}(y0)$	$X0 == \text{someValueFrom}(y0)$ $\&\& x0 > y0 + 10$

Ejecución Simbólica Dinámica (DSE)

```
public static int foo(int v) {  
    return someValueFrom(v);  
}  
  
public static void test_me(int x, int y) {  
    int z = foo(y);  
  
    if (z == x) {  
        if (x > y+10) {  
            // ERROR;  
        }  
    }  
}
```

Estado concreto

Estado simbólico

Condición Camino

Ejecución Simbólica Dinámica (DSE)

```
public static int foo(int v) {  
    return someValueFrom(v);  
}
```

```
public static void test_me(int x, int y) {  
    int z = foo(y);  
  
    if (z == x) {  
        if (x > y+10) {  
            // ERROR;  
        }  
    }  
}
```

test_me(5467,7)

Estado concreto

Estado simbólico

Condición Camino

Ejecución Simbólica Dinámica (DSE)

```
public static int foo(int v) {  
    return someValueFrom(v);  
}
```

```
public static void test_me(int x, int y) {  
    int z = foo(y);  
  
    if (z == x) {  
        if (x > y+10) {  
            // ERROR;  
        }  
    }  
}
```

test_me(5467,7)

Estado concreto	Estado simbólico	Condición Camino
x = 5467, y = 7, z = 5467		

Ejecución Simbólica Dinámica (DSE)

```
public static int foo(int v) {  
    return someValueFrom(v);  
}
```

```
public static void test_me(int x, int y) {  
    int z = foo(y);  
  
    if (z == x) {  
        if (x > y+10) {  
            // ERROR;  
        }  
    }  
}
```

test_me(5467,7)

Estado concreto	Estado simbólico	Condición Camino
$x = 5467, y = 7, z = 5467$	$x = x0, y = y0,$ $z0 = \text{someValueFrom}(y0)$	

Ejecución Simbólica Dinámica (DSE)

```
public static int foo(int v) {  
    return someValueFrom(v);  
}
```

```
public static void test_me(int x, int y) {  
    int z = foo(y);  
  
    if (z == x) {  
        if (x > y+10) {  
            // ERROR;  
        }  
    }  
}
```

test_me(5467,7)

Estado concreto	Estado simbólico	Condición Camino
$x = 5467, y = 7, z = 5467$	$x = x0, y = y0,$ $z0 = \text{someValueFrom}(y0)$	$X0 == \text{someValueFrom}(y0)$ $\&\& x0 > y0 + 10$

Ejecución Simbólica Dinámica (DSE)

```
public static int foo(int v) {  
    return someValueFrom(v);  
}  
  
public static void test_me(int x, int y) {  
    int z = foo(y);  
  
    if (z == x) {  
        if (x > y+10) {  
            // ERROR;  
        }  
    }  
}
```

test_me(5467,7)

CRASH!

Estado concreto	Estado simbólico	Condición Camino
$x = 5467, y = 7, z = 5467$	$x = x0, y = y0,$ $z0 = \text{someValueFrom}(y0)$	$X0 == \text{someValueFrom}(y0)$ $\&\& x0 > y0 + 10$

Heaps Constraints

Qué sucede cuando las entradas son de tipos complejos?

```
void test(Stack stack, int a) {  
    Assume.that(!stack.isFull());  
    Assume.that(stack!=null);  
  
    stack.push(a);  
    int r = stack.pop();  
  
    assertThat(a, is(r));  
}
```


Heaps Constraints

Qué sucede cuando las entradas son de tipos complejos?

```
void test(Stack stack, int a) {  
    Assume.that(!stack.isFull());  
    Assume.that(stack!=null);  
  
    stack.push(a);  
    int r = stack.pop();  
  
    assertThat(a, is(r));  
}
```

- Tendremos Constraints que predican sobre el heap, ej: List0.header!=null

Heaps Constraints

Qué sucede cuando las entradas son de tipos complejos?

```
void test(Stack stack, int a) {  
    Assume.that(!stack.isFull());  
    Assume.that(stack!=null);  
  
    stack.push(a);  
    int r = stack.pop();  
  
    assertThat(a, is(r));  
}
```

- Tendremos Constraints que predican sobre el heap, ej: List0.header!=null

Extendemos el lenguaje de constraints para expresar expresiones sobre direcciones de memoria:

- List0.header!=null
- List0.header.next!=null

Heaps Constraints

Qué sucede cuando las entradas son de tipos complejos?

```
void test(Stack stack, int a) {  
    Assume.that(!stack.isFull());  
    Assume.that(stack!=null);  
  
    stack.push(a);  
    int r = stack.pop();  
  
    assertThat(a, is(r));  
}
```


Heaps Constraints

Qué sucede cuando las entradas son de tipos complejos?

```
void test(Stack stack, int a) {  
    Assume.that(!stack.isFull());  
    Assume.that(stack!=null);  
  
    stack.push(a);  
    int r = stack.pop();  
  
    assertThat(a, is(r));  
}
```

- Tendremos Constraints que predican sobre el heap, ej: List0.header!=null

Heaps Constraints

Qué sucede cuando las entradas son de tipos complejos?

```
void test(Stack stack, int a) {  
    Assume.that(!stack.isFull());  
    Assume.that(stack!=null);  
  
    stack.push(a);  
    int r = stack.pop();  
  
    assertThat(a, is(r));  
}
```

- Tendremos Constraints que predicen sobre el heap, ej: List0.header!=null

Extendemos el lenguaje de constraints para expresar expresiones sobre direcciones de memoria:

- List0.header!=null
- List0.header.next!=null

Var0!=null

Var0.header!=Var0

Var0.header!=null

Var0.header.next!=null

Var0.header.next.next==null



Tools

- IntelliTest: .NET Framework
- JCUTE: Java
- KLEE: C

...

Tools

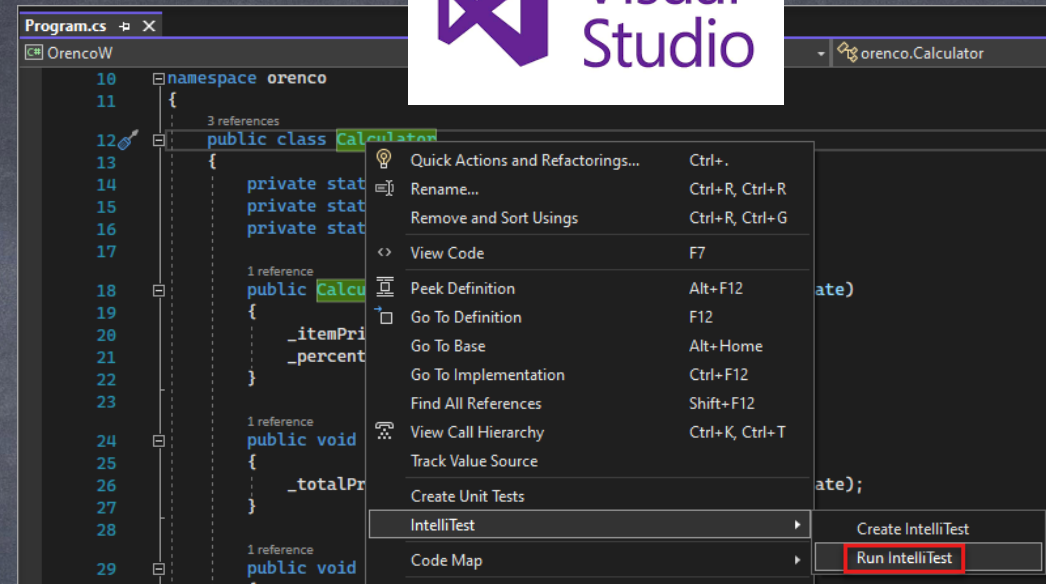
- IntelliTest: .NET Framework

<https://learn.microsoft.com/en-us/visualstudio/test/intellitest-manual/?view=vs-2022>

- JCUTE: Java

- KLEE: C

...



- IntelliTest: .NET Framework

<https://learn.microsoft.com/en-us/visualstudio/test/intellitest-manual/?view=vs-2022>

- JCUTE: Java

- KLEE: C